



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Dipartimento di Informatica - Scienza e Ingegneria

Corso di Laurea in Ingegneria e Scienze Informatiche

Modelli di Illuminazione e Shading in Tempo Reale con Unity: Modelli Classici e Physically Based Rendering

Relatore:
Chiar.ma Prof.ssa
Damiana Lazzaro

Presentata da:
Ciro Bassi

Sessione Unica
Anno Accademico 2025/2026

dedica

Indice

1	La pipeline di rendering	3
1.1	La pipeline di rendering programmabile	3
1.2	Stadi principali della pipeline	4
1.2.1	Vertex shader	5
1.2.2	Rasterizzazione	6
1.2.3	Fragment shader	7
1.2.4	Output merger	7
1.3	La pipeline di rendering in Unity	8
1.4	Il linguaggio HLSL e ShaderLab	8
2	Modelli classici di illuminazione e tecniche di shading	11
2.1	Interazione tra luce e superficie	12
2.1.1	Componenti della luce riflessa	12
2.1.2	Sorgenti luminose	13
2.2	Il modello di Phong	13
2.2.1	Componente ambientale	14
2.2.2	Componente diffusa	14
2.2.3	Componente speculare	15
2.2.4	Formula completa	15
2.2.5	Limiti del modello	15
2.3	Il modello di Blinn-Phong	16
2.3.1	L'halfway vector	16
2.3.2	Formula completa	17

2.3.3	Vantaggi rispetto al modello di Phong	17
2.3.4	Limiti del modello	18
2.4	Tecniche di shading	18
2.4.1	Flat shading	18
2.4.2	Gouraud shading	19
2.4.3	Phong shading	19
2.5	Implementazione HLSL delle tecniche di shading	20
2.5.1	Flat shading	20
2.5.2	Gouraud shading	20
2.5.3	Phong shading	21
2.6	Implementazione HLSL dei modelli classici	22
2.6.1	Struttura comune	23
2.6.2	Modello di Phong	25
2.6.3	Modello di Blinn-Phong	25
3	Physically Based Rendering (PBR)	27
3.1	Principi fisici	28
3.1.1	Conservazione dell'energia	28
3.1.2	Teoria delle microfacce	28
3.1.3	Riflessione e rifrazione	29
3.2	La BRDF di Cook-Torrance	29
3.2.1	Struttura della BRDF	30
3.2.2	Termine D (normal distribution function)	31
3.2.3	Termine F (Fresnel)	32
3.2.4	Termine G (geometry function)	35
3.3	Metallic/Roughness workflow	37
3.3.1	Parametri di un materiale	38
3.3.2	Texture maps	38
3.4	Image-Based Lighting	40
3.5	Implementazione HLSL del modello di Cook-Torrance	42
3.5.1	Struttura dello shader	42
3.5.2	Termine D	43
3.5.3	Termine F	43

3.5.4	Termine G	43
3.5.5	Fragment shader	44
3.5.6	Confronto tra Blinn-Phong e Cook-Torrance	45
3.5.7	Supporto alle texture	47
4	Shader personalizzati per effetti non fotorealistici	51
4.1	Toon shading	51
4.1.1	Componente diffusa quantizzata	52
4.1.2	Componente speculare con soglia	52
4.1.3	Linea di contorno	52
4.1.4	Implementazione HLSL	52
4.2	Hologram shader	55
4.2.1	Rim lighting	55
4.2.2	Pattern a strisce	56
4.2.3	Flicker	56
4.2.4	Trasparenza	56
4.2.5	Implementazione HLSL	57
5	Progetto dimostrativo in Unity	59
5.1	Struttura della scena	59
5.2	Navigazione in prima persona	60
5.3	Selezione degli oggetti	61
5.4	Pannello selezione dello shader	62
5.5	Confronto visivo tra i modelli di illuminazione	64
5.6	Build dell'applicazione	66
6	Conclusioni	67
6.1	Riepilogo dei risultati	67
6.2	Limiti del lavoro	68
6.3	Possibili sviluppi futuri	68

Elenco delle figure

1.1	La sequenza di trasformazioni delle coordinate	6
1.2	I principali stadi della pipeline di rendering	7
1.3	L'Inspector di Unity	10
2.1	Le componenti della luce riflessa	13
2.2	Vettori coinvolti nel modello di Phong	14
2.3	Vettori coinvolti nel modello di Blinn-Phong	17
2.4	Confronto tra le tre tecniche di shading	22
2.5	Confronto tra i modelli di Phong e Blinn-Phong	26
2.6	Confronto tra i riflessi speculari con luce radente	26
3.1	Rappresentazione delle microfacce.	29
3.2	Visualizzazione del termine D	32
3.3	Effetto Fresnel su un lago	33
3.4	Visualizzazione del termine F	35
3.5	Effetti di masking e shadowing	36
3.6	Visualizzazione del termine G	37
3.7	Le texture map	40
3.8	Confronto tra luce diretta e indiretta	41
3.9	Cubemap	42
3.10	Sfere con rugosità crescente illuminate tramite IBL	42
3.11	Confronto tra i modelli di Blinn-Phong e di Cook-Torrance	46
3.12	Effetto del parametro metallic	46
3.13	Confronto su un materiale metallico	47
3.14	Confronto con rugosità elevata	47

3.15	Confronto tra valori costanti e texture maps	49
4.1	I due pass del toon shading	54
4.2	Effetto del numero di fasce nel toon shading	54
4.3	Confronto tra Blinn-Phong e toon shading	55
4.4	Effetto ologramma applicato a un oggetto	58
5.1	Panoramica della scena dimostrativa	60
5.2	Il pannello di selezione dello shader	64
5.3	Lo stesso oggetto con i sei shader implementati	65

Elenco dei codici

1.1	Struttura minima di uno shader custom per URP.	9
2.1	Implementazione del flat shading.	20
2.2	Implementazione del Gouraud shading.	21
2.3	Implementazione del Phong shading.	22
2.4	Struttura comune degli shader per i modelli classici.	24
2.5	Fragment shader del modello di Phong.	25
2.6	Fragment shader del modello di Blinn-Phong.	25
3.1	Implementazione del termine D.	43
3.2	Implementazione del termine F.	43
3.3	Implementazione del termine G.	44
3.4	Fragment shader del modello Cook-Torrance.	45
3.5	Lettura dei parametri da texture.	48
3.6	Applicazione della normal map (fragment shader).	48
4.1	Primo pass: estrude i vertici lungo la normale e disegna solo le facce posteriori.	53
4.2	Fragment shader del secondo pass: la componente diffusa viene quan- tizzata, mentre alla speculare viene applicata una soglia	53
4.3	Configurazione per la trasparenza.	56
4.4	Fragment shader dell'effetto ologramma.	57
5.1	Logica di movimento e rotazione della camera.	61
5.2	Creazione delle istanze dei materiali e applicazione.	62
5.3	Selezione di un oggetto tramite raycast.	62
5.4	Esempio di generazione di un controllo del pannello.	63

Introduzione

La sintesi di immagini tridimensionali fotorealistiche è da sempre uno degli obiettivi principali della computer graphics. Alla base di qualsiasi immagine generata in tempo reale c'è la *pipeline di rendering*: un processo complesso che coinvolge la geometria della scena, le proprietà dei materiali e il comportamento della luce.

Negli ultimi decenni, l'evoluzione dell'hardware grafico ha reso possibile eseguire questo processo a frequenze interattive, migliorando l'esperienza in applicazioni video-ludiche e non solo. Parallelamente, i modelli matematici per descrivere l'interazione tra luce e materia sono stati raffinati: dai modelli empirici di Phong e Blinn-Phong fino ai moderni modelli di *Physically Based Rendering* (PBR), che approssimano più fedelmente il comportamento della luce.

La scelta del modello di illuminazione ha un impatto diretto sulla qualità visiva e sulle prestazioni di un'applicazione in tempo reale. Un modello empirico come Blinn-Phong è computazionalmente economico e intuitivo, ma produce risultati che si discostano sensibilmente dalla realtà. Il PBR, al contrario, garantisce una risposta visiva coerente in qualsiasi contesto, a fronte di un costo computazionale più elevato. Comprendere le differenze tra questi approcci è fondamentale per operare nel campo della grafica interattiva.

L'obiettivo di questo elaborato è comparare i principali modelli di illuminazione utilizzati nella grafica in tempo reale, implementandoli in HLSL all'interno del motore Unity. A tal fine è stata adottata la *Universal Render Pipeline* (URP), una pipeline leggera e versatile che consente di scrivere shader personalizzati con pieno controllo sul calcolo dell'illuminazione, rendendola ideale per implementare e confrontare

direttamente i modelli trattati. Il lavoro copre la pipeline di rendering programmabile, i modelli classici di Phong e Blinn-Phong, il Physically Based Rendering con la BRDF di Cook-Torrance, alcuni shader personalizzati a scopo dimostrativo e un progetto in Unity che mette a confronto visivamente i modelli implementati.

La tesi è così strutturata:

- il Capitolo 1 introduce la pipeline di rendering programmabile nei suoi stadi principali e fornisce le basi di Unity e del linguaggio HLSL;
- il Capitolo 2 tratta i modelli di illuminazione classici: vengono analizzati i modelli di Phong e Blinn-Phong, le diverse tecniche di shading (flat, Gouraud e Phong shading) e le relative implementazioni HLSL;
- il Capitolo 3 è dedicato al Physically Based Rendering: vengono illustrati i principi fisici alla base del modello, la BRDF di Cook-Torrance con la relativa implementazione HLSL, il workflow metallic/roughness e un'introduzione all'Image-Based Lighting;
- il Capitolo 4 presenta alcuni shader personalizzati che esulano dal fotorealismo, mostrando le possibilità espressive della pipeline programmabile;
- il Capitolo 5 descrive il progetto dimostrativo realizzato in Unity, analizzando le differenze visive e prestazionali tra i modelli implementati;
- il Capitolo 6 raccoglie le considerazioni conclusive.

Capitolo 1

La pipeline di rendering

La computer graphics ha come obiettivo principale la generazione di immagini a partire da una descrizione matematica di una scena tridimensionale. Tale descrizione, costituita da geometrie, materiali, luci e parametri della camera, viene trasformata progressivamente in un'immagine bidimensionale destinata alla visualizzazione su schermo. L'insieme delle fasi che realizzano questa trasformazione prende il nome di *pipeline di rendering*. Nelle architetture moderne, gran parte di tale processo è implementata direttamente in hardware dalle GPU (*Graphics Processing Unit*), che eseguono in modo altamente parallelo le operazioni necessarie alla produzione dei pixel finali.

Questo capitolo introduce la struttura generale della pipeline grafica programmabile, descrivendo i suoi stadi principali e il loro ruolo nel processo di rendering.

1.1 La pipeline di rendering programmabile

Le prime GPU erano basate su una pipeline non programmabile (*fixed-function pipeline*): ogni stadio del processo di rendering eseguiva operazioni rigidamente predefinite a livello hardware, senza alcuna possibilità di personalizzazione da parte dello sviluppatore. Questo approccio rendeva l'implementazione di effetti grafici non standard estremamente complessa, se non del tutto impossibile.

A partire dai primi anni 2000, con l'introduzione delle prime GPU programmabili, alcune fasi della pipeline divennero configurabili e personalizzabili dallo sviluppatore. Le GPU iniziarono a integrare unità computazionali dedicate, dette *Shader Units*, riprogrammabili tramite appositi programmi chiamati *shader*. Gli shader, scritti direttamente dallo sviluppatore, vengono eseguiti in parallelo sulla GPU per ogni vertice o frammento della scena. Questo approccio ha reso possibile implementare algoritmi grafici personalizzati, superando i limiti imposti dalla pipeline non programmabile.

La pipeline grafica moderna, ottimizzata nella quasi totalità dei casi per l'elaborazione di triangoli, è composta da una sequenza di stadi che possono essere raggruppati in tre fasi concettuali:

1. **Fase applicativa:** eseguita sulla CPU, si occupa di preparare i dati della scena (geometrie, parametri dei materiali, ...) e di inviarli alla GPU.
2. **Fase geometrica:** opera sui vertici della scena, trasformandone le coordinate dallo spazio locale dell'oggetto a quello del mondo, fino allo spazio di proiezione della camera. Comprende il *vertex shader* e, opzionalmente, altri stadi tra cui *tessellation* e *geometry shader*.
3. **Fase di rasterizzazione e shading:** converte le primitive geometriche in frammenti (potenziali pixel), ne calcola il colore finale attraverso il *fragment shader*, e determina quali frammenti vengono effettivamente visualizzati a schermo tramite l'*output merger*.

1.2 Stadi principali della pipeline

Di seguito vengono descritti i principali stadi della pipeline. Tra questi, il vertex shader e il fragment shader rappresentano le principali fasi programmabili: è infatti al loro interno che è possibile definire il comportamento della geometria e calcolare il colore finale dei frammenti.

1.2.1 Vertex shader

Il *vertex shader* è il primo stadio programmabile della pipeline. Viene eseguito una volta per ogni vertice della geometria inviata alla GPU e ha come compito principale quello di trasformarne le coordinate attraverso una serie di spazi di riferimento, fino a ottenere la posizione finale utilizzata nelle fasi successive della pipeline:

1. **Object space:** rappresenta lo spazio locale del modello 3D. In questo sistema di riferimento, le coordinate del vertice sono definite rispetto all'origine e all'orientamento dell'oggetto stesso.
2. **World space:** rappresenta lo spazio globale della scena. Le coordinate del vertice vengono trasformate in base alla posizione, alla rotazione e alla scala dell'oggetto, mediante la matrice di modello $\mathbf{M}$.
3. **View space:** rappresenta lo spazio della camera. Le coordinate sono espresse rispetto al punto di vista dell'osservatore, mediante la matrice di vista $\mathbf{V}$.
4. **Clip space:** rappresenta lo spazio ottenuto dopo l'applicazione della proiezione. Le coordinate vengono trasformate mediante la matrice di proiezione $\mathbf{P}$, preparando la geometria alle successive fasi di clipping, divisione prospettica e rasterizzazione.

La trasformazione complessiva di un vertice $\mathbf{v}$ dalla posizione in object space alla posizione in clip space è data dal prodotto:

$$\mathbf{v}_{\text{clip}} = \mathbf{P} \cdot \mathbf{V} \cdot \mathbf{M} \cdot \mathbf{v} \quad (1.1)$$

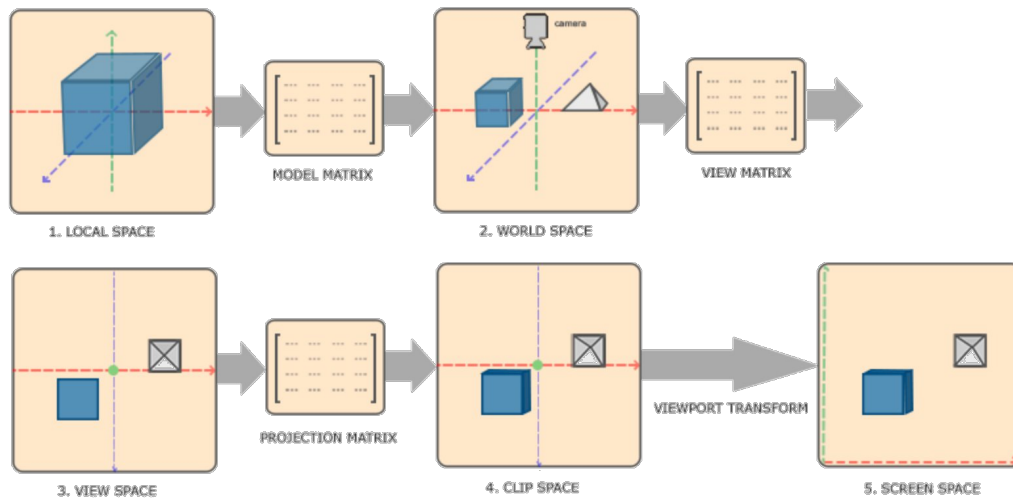


Figura 1.1: La sequenza di trasformazioni delle coordinate: dallo spazio locale dell'oggetto (*object space*), allo spazio del mondo (*world space*), allo spazio della camera (*view space*), fino allo spazio di proiezione (*clip space*).

Oltre alla posizione, il vertex shader può ricevere in ingresso e trasformare altri attributi associati al vertice, come la normale $\mathbf{N}$, fondamentale per i calcoli di illuminazione, o le coordinate texture, necessarie per il campionamento delle immagini applicate alla superficie.

I dati prodotti dal vertex shader, detti *varyings*, vengono successivamente interpolati durante la fase di rasterizzazione prima di essere forniti in ingresso al fragment shader.

1.2.2 Rasterizzazione

La rasterizzazione è lo stadio, non programmabile, che converte le primitive geometriche (triangoli definiti dai loro vertici in clip space, ottenuti dal vertex shader) in un insieme di frammenti, corrispondenti ciascuno a un potenziale pixel sullo schermo.

Per ogni frammento generato, la rasterizzazione interpola linearmente gli attributi calcolati dal vertex shader nei vertici del triangolo corrispondente.

1.2.3 Fragment shader

Il *fragment shader* è il secondo dei principali stadi programmabili della pipeline. Esso viene eseguito una volta per ciascun frammento generato dalla rasterizzazione e ha il compito di determinarne il colore finale. Per svolgere questa operazione, il fragment shader riceve in ingresso gli attributi interpolati durante la fase di rasterizzazione, come la normale, la posizione nello spazio e le coordinate texture. Tali informazioni vengono utilizzate per valutare il modello di illuminazione scelto e calcolare il contributo della luce sulla superficie. I modelli di illuminazione vengono solitamente implementati in questo stadio, anziché nel vertex shader, poiché il calcolo dell'illuminazione per-pixel produce risultati significativamente più accurati rispetto al calcolo per-vertex, soprattutto in presenza di riflessi speculari o variazioni locali della superficie.

1.2.4 Output merger

L'*output merger* è l'ultimo stadio della pipeline, non programmabile. Riceve i frammenti prodotti dal fragment shader e decide quali vengono effettivamente scritti nel framebuffer (il buffer che contiene l'immagine finale destinata allo schermo). Questa decisione avviene attraverso una serie di operazioni, tra cui il *depth test*, che scarta i frammenti nascosti da geometrie più vicine alla camera, e il *blending*, che combina il colore del frammento con quello già presente nel framebuffer. Quest'ultima operazione è particolarmente importante nella gestione della trasparenza.

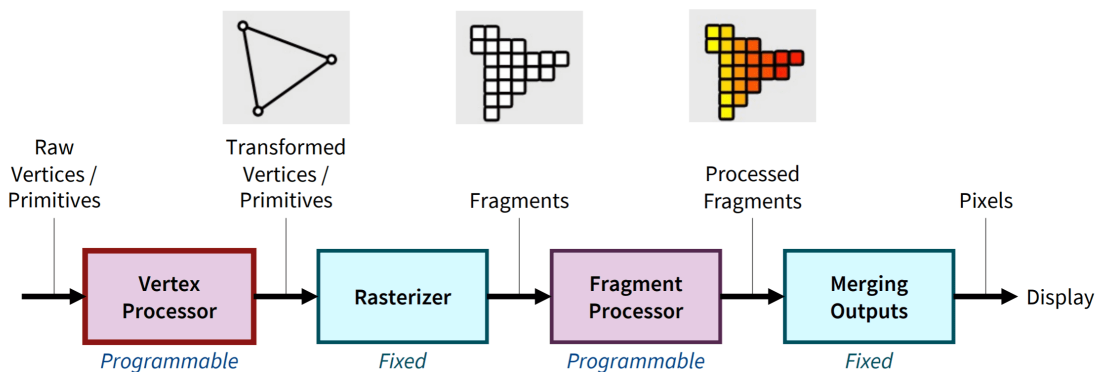


Figura 1.2: I principali stadi della pipeline di rendering.

1.3 La pipeline di rendering in Unity

Unity è un motore di gioco che supporta diverse pipeline di rendering attraverso il sistema *Scriptable Render Pipeline* (SRP). Questo consente di definire l'intero flusso di rendering, offrendo un livello di controllo che la precedente pipeline integrata (*Built-in Render Pipeline*) non permetteva.

Le due pipeline attualmente fornite da Unity sono la *Universal Render Pipeline* (URP), orientata alle prestazioni e alla portabilità, e la *High Definition Render Pipeline* (HDRP), orientata alla massima qualità visiva su hardware di fascia alta.

Nel progetto sviluppato in questa tesi è stata adottata URP: poiché l'obiettivo principale consiste nell'implementazione manuale dei modelli di illuminazione, era necessario utilizzare una pipeline che offrisse un controllo diretto sul codice di rendering, riducendo al minimo la complessità aggiuntiva. In questo modo è stato possibile concentrare l'attenzione sulla logica di illuminazione e sul confronto tra i modelli implementati. La scelta della pipeline non influenza i fondamenti teorici trattati: i modelli, le tecniche di shading e la struttura della pipeline grafica rimangono concettualmente identici indipendentemente dalla pipeline adottata.

1.4 Il linguaggio HLSL e ShaderLab

Gli shader in Unity vengono scritti in file con estensione `.shader` che utilizzano ShaderLab, un linguaggio proprietario che descrive la struttura generale dello shader, le sue proprietà e i pass di rendering. All'interno di questa struttura viene inserito il codice HLSL, responsabile dell'implementazione della logica del vertex shader e del fragment shader.

Uno shader Unity è organizzato nei seguenti blocchi: **Properties**, che espone i parametri configurabili del materiale dell'inspector di Unity; **Shader**, il blocco esterno identificato da un nome; uno o più **SubShader**, ciascuno pensato per una specifica piattaforma; uno o più **Pass**, che rappresentano singole esecuzioni della pipeline, ognuna con il proprio *vertex* e *fragment shader*.

L'esempio seguente mostra la struttura minima di uno shader per URP, con un vertex shader che trasforma le coordinate e un fragment shader che restituisce un colore uniforme:

Listing 1.1: Struttura minima di uno shader custom per URP.

```

1  Shader "Custom/NomeShader" {
2      Properties { // Proprieta' configurabili dall'inspector di Unity
3          _Color ("Color", Color) = (1, 1, 1, 1)
4      }
5      SubShader {
6          Tags { "RenderPipeline"="UniversalPipeline" }
7          Pass {
8              Tags { "LightMode"="UniversalForward" }
9              HLSLPROGRAM // Inizio del codice HLSL
10             #pragma vertex vert
11             #pragma fragment frag
12             #include
13
14                 "Packages/com.unity.render-pipelines.universal/ShaderLibrary/Core.hlsl"
15
16             struct Attributes { // Dati in input al vertex shader
17                 float4 positionOS : POSITION;
18                 float3 normalOS   : NORMAL;
19             };
20             struct Varyings { // Dati in input al fragment shader (interpolati)
21                 float4 positionCS : SV_POSITION;
22                 float3 normalWS   : TEXCOORD0;
23                 float3 positionWS : TEXCOORD1;
24             };
25
26             float4 _Color; // rende la property accessibile al codice HLSL
27
28             Varyings vert(Attributes IN) { // Vertex shader
29                 Varyings OUT;
30                 OUT.positionCS = TransformObjectToHClip(IN.positionOS.xyz);
31                 OUT.normalWS   = TransformObjectToWorldNormal(IN.normalOS);
32                 OUT.positionWS = TransformObjectToWorld(IN.positionOS.xyz);
33                 return OUT;
34             }
35
36             float4 frag(Varyings IN) : SV_Target { // Fragment shader
37                 float3 normal = normalize(IN.normalWS);
38                 return float4(_Color.rgb, 1.0);
39             }
40             ENDHLSL // Fine del codice HLSL
41         }
42     }
43 }

```

Le strutture `Attributes` e `Varyings` definiscono rispettivamente i dati in ingresso al *vertex shader* e quelli passati al fragment shader dopo la rasterizzazione. Ogni campo è annotato con una **semantica** (`POSITION`, `NORMAL`, `SV_POSITION`, ...) che indica alla GPU il significato del dato e come instradarlo nella pipeline. I suffissi `OS`, `WS` e `CS` sono una convenzione di Unity per indicare lo spazio di riferimento del valore: *Object Space*, *World Space* e *Clip Space*.

Questa struttura è la base su cui verranno costruiti tutti gli shader nei capitoli successivi.

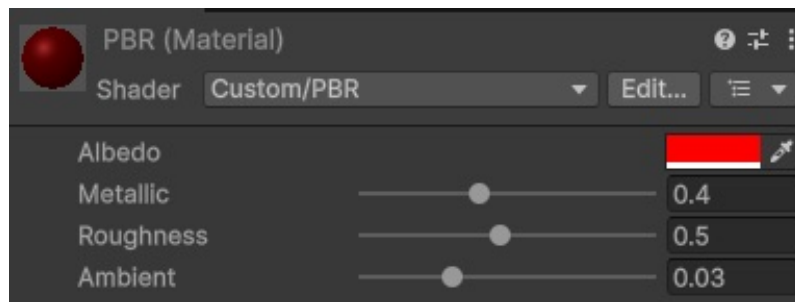


Figura 1.3: L'inspector di un materiale in Unity: i parametri mostrati corrispondono alle proprietà dichiarate nel blocco `Properties` dello shader.

Capitolo 2

Modelli classici di illuminazione e tecniche di shading

Questo capitolo presenta i modelli di illuminazione storicamente più rilevanti nella grafica in tempo reale: il modello di Phong e la sua variante proposta da Blinn. Si tratta di modelli empirici, pur non essendo fondati su una descrizione fisicamente accurata dell'interazione tra luce e materia, approssimano in modo efficace il comportamento visivo della luce, mantenendo un costo computazionale contenuto.

Entrambi sono modelli di **illuminazione locale**: calcolano il colore di un punto considerando solo la luce che arriva direttamente dalle sorgenti, senza tener conto dei rimbalzi successivi tra le superfici. Al contrario, i modelli di **illuminazione globale** simulano anche la luce indiretta producendo risultati molto più realistici, ma a un costo computazionale spesso incompatibile con il rendering in tempo reale e non verranno trattati in questa tesi. Il PBR, trattato nel Capitolo 3, può avvicinarsi all'illuminazione globale tramite l'Image-Based Lighting, pur restando nell'ambito dei modelli locali.

Dopo aver introdotto i concetti di base sull'interazione tra luce e superficie, vengono formalizzati i due modelli e analizzate le differenze tra le principali tecniche di shading: flat, Gouraud e Phong shading. Il capitolo si conclude con le implementazioni HLSL dei modelli di illuminazione in Unity.

2.1 Interazione tra luce e superficie

Per comprendere i modelli di illuminazione è necessario introdurre i concetti fisici alla base dell'interazione tra luce e superficie, analizzando sia il comportamento della luce quando incide su un materiale, sia il modo in cui la risposta della superficie può essere scomposta in componenti più semplici da modellare.

2.1.1 Componenti della luce riflessa

Quando un raggio di luce colpisce una superficie opaca, la sua energia viene in parte assorbita e in parte riflessa. La componente riflessa può essere descritta come la somma di tre contributi distinti:

- **Componente ambientale:** rappresenta la luce indiretta, ovvero quella che ha rimbalzato più volte nell'ambiente circostante prima di raggiungere la superficie. Nei modelli classici viene approssimata con un termine costante: una stima grossolana, ma evita che le zone in ombra appaiano completamente nere.
- **Componente diffusa:** rappresenta la luce riemessa dalla superficie uniformemente in tutte le direzioni. La sua intensità non dipende dalla posizione dell'osservatore, bensì dall'angolo tra la direzione della luce incidente e la normale alla superficie.
- **Componente speculare:** rappresenta la luce riflessa direzionalmente dalla superficie. A differenza della componente diffusa, la sua intensità dipende anche dalla posizione dell'osservatore: il riflesso speculare è visibile solo quando l'occhio si trova in prossimità della direzione di riflessione. La forma e l'intensità di questo riflesso dipendono dalle caratteristiche del materiale: un metallo lucido produce un riflesso piccolo e intenso, mentre una plastica opaca ne produce uno ampio e debole.

Il colore finale di un punto sulla superficie viene calcolato come la somma delle tre componenti:

$$I = I_a + I_d + I_s \quad (2.1)$$

dove I_a , I_d e I_s rappresentano rispettivamente le componenti ambientale, diffusa e speculare.

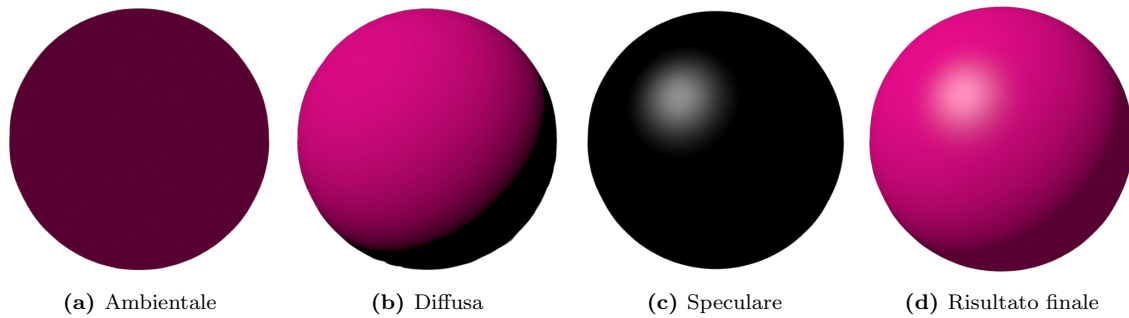


Figura 2.1: Le tre componenti della luce riflessa su una sfera e la loro somma.

2.1.2 Sorgenti luminose

Esistono diverse tipologie di sorgenti luminose (direzionali, puntuali e spot) che si differenziano per geometria e comportamento. Negli shader sviluppati successivamente verrà utilizzata, per semplicità, esclusivamente una luce direzionale, che modella una sorgente infinitamente lontana con raggi luminosi paralleli, una tipica approssimazione della luce solare.

2.2 Il modello di Phong

Il modello di illuminazione di Phong, è un modello di illuminazione locale proposto da Bui Tuong Phong nel 1975^[Pho75], è uno dei primi modelli empirici per la sintesi di immagini tridimensionali. Pur non essendo fisicamente accurato, produce risultati visivamente convincenti a costo computazionale contenuto, ed è rimasto a lungo il riferimento principale nella grafica in tempo reale.

Il modello si basa sulla decomposizione della luce riflessa nelle tre componenti introdotte nella Sezione 2.1.1: ambientale, diffusa e speculare.

Per comprenderne la formulazione matematica è necessario definire i principali vettori unitari coinvolti nel calcolo dell'illuminazione, valutati nel punto della superficie considerato:

- **N**: normale alla superficie, diretta verso l'esterno;
- **L**: direzione verso la sorgente luminosa;
- **V**: direzione verso l'osservatore;
- **R**: direzione di riflessione di **L** rispetto a **N**, calcolata come:

$$\mathbf{R} = 2(\mathbf{N} \cdot \mathbf{L}) \mathbf{N} - \mathbf{L} \quad (2.2)$$

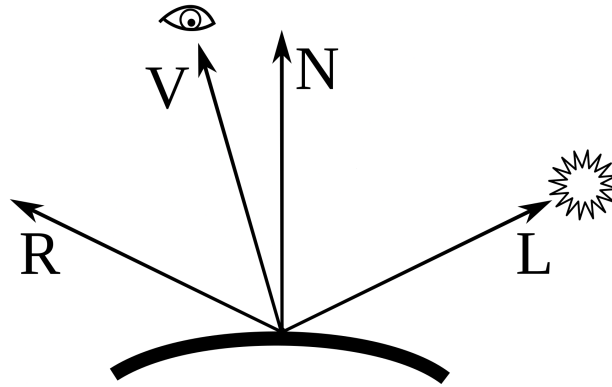


Figura 2.2: Vettori coinvolti nel modello di Phong. Immagine adattata da ^[Wik].

2.2.1 Componente ambientale

L'intensità della componente ambientale è data da:

$$I_a = k_a \cdot c \quad (2.3)$$

dove $k_a \in [0, 1]$ è il coefficiente di riflessione ambientale e c è il colore del materiale.

2.2.2 Componente diffusa

L'intensità della componente diffusa segue la legge di Lambert: dipende dal coseno dell'angolo tra **N** e **L**. Poiché entrambi i vettori sono unitari, il coseno corrisponde

al loro prodotto scalare:

$$I_d = k_d \cdot (\mathbf{N} \cdot \mathbf{L}) \cdot c \quad (2.4)$$

dove $k_d \in [0, 1]$ è il coefficiente di riflessione diffusa e c è il colore del materiale. Nell'implementazione il prodotto scalare viene limitato a zero per evitare valori negativi quando la superficie è rivolta in direzione opposta alla luce.

2.2.3 Componente speculare

L'intensità della componente speculare dipende dall'angolo tra la direzione di riflessione $\mathbf{R}$ e la direzione di vista $\mathbf{V}$:

$$I_s = k_s \cdot (\mathbf{R} \cdot \mathbf{V})^n \cdot c_L \quad (2.5)$$

dove $k_s \in [0, 1]$ è il coefficiente di riflessione speculare, n è l'esponente di lucentezza (*shininess*) e c_L è il colore della sorgente luminosa. A differenza delle componenti ambientale e diffusa, quella speculare viene moltiplicata per il colore della luce e non del materiale, poiché il riflesso speculare è luce che rimbalza sulla superficie senza essere assorbita. La shininess n controlla l'ampiezza del riflesso: valori bassi producono riflessi ampi e morbidi, mentre valori alti producono riflessi piccoli e concentrati.

2.2.4 Formula completa

La formula finale del modello di Phong è data dalla somma delle tre componenti:

$$I = I_a + I_d + I_s = k_a \cdot c + k_d \cdot (\mathbf{N} \cdot \mathbf{L}) \cdot c + k_s \cdot (\mathbf{R} \cdot \mathbf{V})^n \cdot c_L \quad (2.6)$$

2.2.5 Limiti del modello

Nonostante la sua semplicità ed efficacia, il modello di Phong presenta alcuni limiti significativi:

- **Non conserva l'energia:** nessun vincolo impedisce che la somma dei coefficienti $k_a + k_d + k_s$ superi 1, producendo più luce riflessa di quella incidente.
- **Il modello è empirico:** le tre componenti non derivano da principi fisici ma da un'osservazione qualitativa del comportamento della luce. Il risultato è visivamente plausibile ma non fisicamente corretto.
- **Riflesso speculare troncato:** quando l'angolo tra $\mathbf{R}$ e $\mathbf{V}$ supera i 90° , il prodotto scalare diventa negativo e viene annullato (poiché limitato a zero). In alcuni casi questo produce un taglio netto del riflesso speculare (Figura 2.6a).

L'ultimo di questi limiti è uno dei motivi principali dell'introduzione del modello di Blinn-Phong.

2.3 Il modello di Blinn-Phong

Nel 1977 James Blinn^[Bli77] propose una variante del modello di Phong che ne modifica la componente speculare, mantenendo invariate le componenti ambientale e diffusa.

2.3.1 L'halfway vector

La modifica introdotta da Blinn consiste nel sostituire il vettore di riflessione $\mathbf{R}$ con l'*halfway vector* $\mathbf{H}$, definito come il vettore a metà tra la direzione della luce $\mathbf{L}$ e la direzione di vista $\mathbf{V}$:

$$\mathbf{H} = \frac{\mathbf{L} + \mathbf{V}}{\|\mathbf{L} + \mathbf{V}\|} \quad (2.7)$$

Si osserva che una superficie riflette la luce in modo speculare verso l'osservatore quando la sua normale è allineata con $\mathbf{H}$. La componente speculare viene quindi riscritta come:

$$I_s = k_s \cdot (\mathbf{N} \cdot \mathbf{H})^n \cdot c_L \quad (2.8)$$

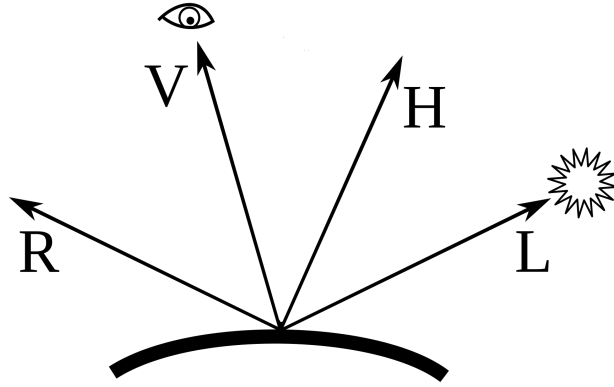


Figura 2.3: Vettori coinvolti nel modello di Blinn-Phong. Immagine adattata da ^[Wik].

Poiché l'angolo tra $\mathbf{N}$ e $\mathbf{H}$ è circa la metà dell'angolo tra $\mathbf{R}$ e $\mathbf{V}$, a parità di esponente n il modello di Blinn-Phong produce un riflesso speculare più ampio rispetto a Phong. Per ottenere un riflesso di dimensione simile è necessario utilizzare un esponente circa quattro volte maggiore.

2.3.2 Formula completa

Le componenti ambientale e diffusa rimangono invariate rispetto al modello di Phong. La formula completa di Blinn-Phong è quindi:

$$I = k_a \cdot c + k_d \cdot (\mathbf{N} \cdot \mathbf{L}) \cdot c + k_s \cdot (\mathbf{N} \cdot \mathbf{H})^n \cdot c_L \quad (2.9)$$

2.3.3 Vantaggi rispetto al modello di Phong

La sostituzione di $\mathbf{R} \cdot \mathbf{V}$ con $\mathbf{N} \cdot \mathbf{H}$ comporta due vantaggi principali.

Il primo è computazionale: il calcolo di $\mathbf{R}$ richiede una riflessione (equazione 2.2), mentre $\mathbf{H}$ richiede solo una somma e una normalizzazione.

Il secondo riguarda la qualità visiva. Il problema del riflesso speculare troncato descritto nella sezione precedente non si presenta con Blinn-Phong: l'angolo tra $\mathbf{N}$ e $\mathbf{H}$ non supera mai i 90° in condizioni normali, eliminando l'artefatto.

Per queste ragioni Blinn-Phong ha sostituito il modello di Phong originale come standard nella grafica in tempo reale.

2.3.4 Limiti del modello

Blinn-Phong risolve il problema del riflesso speculare troncato ma condivide con Phong tutte le altre limitazioni: assenza di conservazione dell'energia, natura empirica dei coefficienti, nessuna distinzione tra materiali metallici e dielettrici.

Queste limitazioni saranno affrontate nel Capitolo 3 con l'introduzione del Physically Based Rendering.

2.4 Tecniche di shading

I modelli di illuminazione descritti nelle sezioni precedenti stabiliscono come calcolare il colore di un punto su una superficie, mentre le tecniche di shading stabiliscono in quali punti della geometria tale modello viene valutato e come i risultati ottenuti vengono distribuiti sulla superficie. La scelta della tecnica di shading è indipendente dal modello di illuminazione adottato.

2.4.1 Flat shading

Nel *flat shading* il modello di illuminazione viene valutato una sola volta per faccia, utilizzando la normale del poligono, e il colore risultante viene assegnato uniformemente all'intera faccia senza alcuna interpolazione. Il risultato è un'immagine in cui i singoli poligoni sono chiaramente distinguibili, il che lo rende inadatto alla rappresentazione di superfici lisce.

Dal punto di vista implementativo, il flat shading si ottiene assegnando a ogni vertice di un poligono la stessa normale, pari a quella della faccia, senza condividerla con le facce adiacenti.

2.4.2 Gouraud shading

Il *Gouraud shading*, proposto da Henri Gouraud nel 1971^[Gou71], valuta il modello di illuminazione una volta per vertice. Il colore calcolato in ciascun vertice viene poi interpolato sulla superficie del poligono durante la rasterizzazione. Per ottenere transizioni più morbide tra facce adiacenti, la normale associata a ciascun vertice non coincide generalmente con la normale di una singola faccia, ma viene calcolata come media delle normali delle facce che condividono quel vertice. In questo modo si ottiene una variazione più continua dell'illuminazione sulla superficie, riducendo l'aspetto sfaccettato tipico del flat shading.

Rispetto al flat shading, il risultato è una superficie con variazioni di colore continue, che elimina le discontinuità tra facce adiacenti. Tuttavia l'interpolazione del colore presenta un limite significativo: i riflessi speculari vengono calcolati solo nelle posizioni dei vertici. Se un riflesso cade al centro di un poligono, lontano dai vertici, il Gouraud shading non è in grado di catturarlo. Il riflesso risulta quindi assente o fortemente distorto.

2.4.3 Phong shading

Il *Phong shading*, proposto nello stesso articolo del modello di illuminazione^[Pho75], risolve i limiti del Gouraud shading con un approccio diverso: anziché interpolare il colore tra i vertici, interpola le normali. Il modello di illuminazione viene poi valutato una volta per frammento, utilizzando la normale interpolata. Poiché l'interpolazione non ne preserva la lunghezza unitaria, è necessario rinormalizzarla prima di procedere con i calcoli dell'illuminazione.

Questo approccio produce riflessi speculari corretti indipendentemente dalla densità della mesh, poiché il calcolo avviene in ogni pixel e non solo nei vertici. Il costo computazionale è più alto (il modello di illuminazione viene valutato per ogni frammento anziché per ogni vertice) ma sulle GPU moderne è una differenza trascurabile.

Tutti gli shader sviluppati in seguito utilizzeranno il Phong shading, poiché è l'unico che produce risultati accurati in presenza di riflessi speculari.

2.5 Implementazione HLSL delle tecniche di shading

Le tre tecniche di shading si distinguono per il punto in cui viene valutato il modello di illuminazione e per i dati scambiati tra *vertex* e *fragment shader*, che rimane comunque indipendente dalla tecnica scelta.

2.5.1 Flat shading

Nel flat shading ogni frammento di un triangolo riceve lo stesso colore, senza interpolazione. In HLSL questo si ottiene calcolando l'illuminazione nel vertex shader e annotando il risultato con il modificatore `nointerpolation`, che impedisce alla GPU di interpolare il valore tra i vertici:

Listing 2.1: Implementazione del flat shading.

```
1 struct Varyings {
2     float4 positionCS : SV_POSITION;
3     nointerpolation float3 color : TEXCOORD0; // colore non interpolato
4 };
5
6 Varyings vert(Attributes IN) {
7     Varyings OUT;
8     OUT.positionCS = TransformObjectToHClip(IN.positionOS.xyz);
9     float3 posWS = TransformObjectToWorld(IN.positionOS.xyz);
10    float3 N = normalize(TransformObjectToWorldNormal(IN.normalOS));
11    OUT.color = ... // calcolo dell'illuminazione
12    return OUT;
13 }
14
15 float4 frag(Varyings IN) : SV_Target {
16     return float4(IN.color, 1.0);
17 }
```

2.5.2 Gouraud shading

Nel Gouraud shading il calcolo dell'illuminazione avviene sempre nel vertex shader, ma il colore viene interpolato tra i vertici. L'unica differenza rispetto al flat shading è la rimozione di `nointerpolation`:

Listing 2.2: Implementazione del Gouraud shading.

```
1 struct Varyings {
2     float4 positionCS : SV_POSITION;
3     float3 color : TEXCOORD0; // colore interpolato
4 };
5
6 Varyings vert(Attributes IN) {
7     Varyings OUT;
8     OUT.positionCS = TransformObjectToHClip(IN.positionOS.xyz);
9     float3 posWS = TransformObjectToWorld(IN.positionOS.xyz);
10    float3 N = normalize(TransformObjectToWorldNormal(IN.normalOS));
11    OUT.color = ... // calcolo dell'illuminazione
12    return OUT;
13 }
14
15 float4 frag(Varyings IN) : SV_Target {
16     return float4(IN.color, 1.0);
17 }
```

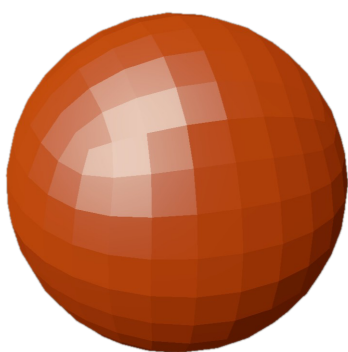
Il fragment shader è identico a quello del flat shading: restituisce il colore senza ulteriori calcoli. La differenza visiva è interamente dovuta all'interpolazione operata dalla rasterizzazione.

2.5.3 Phong shading

Nel Phong shading il calcolo dell'illuminazione viene spostato nel fragment shader. Il vertex shader non calcola più il colore, ma passa la normale e la posizione, che vengono interpolate dalla rasterizzazione. Il fragment shader utilizza la normale interpolata per valutare il modello di illuminazione in ogni frammento:

Listing 2.3: Implementazione del Phong shading.

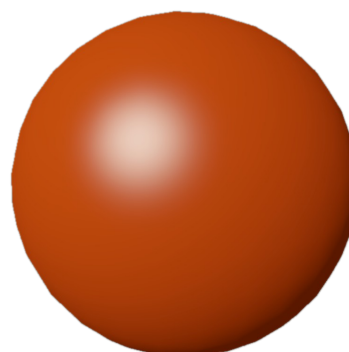
```
1 struct Varyings {
2     float4 positionCS : SV_POSITION;
3     float3 normalWS : TEXCOORD0; // normale interpolata
4     float3 positionWS : TEXCOORD1;
5 };
6
7 Varyings vert(Attributes IN) {
8     Varyings OUT;
9     OUT.positionCS = TransformObjectToHClip(IN.positionOS.xyz);
10    OUT.normalWS = TransformObjectToWorldNormal(IN.normalOS);
11    OUT.positionWS = TransformObjectToWorld(IN.positionOS.xyz);
12    return OUT;
13 }
14
15 float4 frag(Varyings IN) : SV_Target {
16     float3 N = normalize(IN.normalWS);
17     return ... // calcolo dell'illuminazione
18 }
```



(a) Flat shading



(b) Gouraud shading



(c) Phong shading

Figura 2.4: Confronto tra le tre tecniche di shading applicate a una sfera a bassa risoluzione.

2.6 Implementazione HLSL dei modelli classici

In questa sezione vengono presentate le implementazioni HLSL dei modelli di illuminazione trattati in precedenza, realizzate come shader custom per la Universal Render Pipeline di Unity.

2.6.1 Struttura comune

Gli shader condividono la stessa struttura: un vertex shader che trasforma posizione e normale, e un fragment shader che riceve i dati interpolati e vi applica il modello di illuminazione (Phong shading). La differenza tra uno shader e l'altro risiede unicamente nel calcolo della componente speculare.

I parametri del materiale vengono esposti come proprietà nel blocco `Properties`, così da poter essere regolati dall'inspector di Unity senza modificare il codice.

Listing 2.4: Struttura comune degli shader per i modelli classici.

```
1 Shader "Custom/NomeModello" {
2     Properties {
3         _Color ("Colore", Color) = (1, 1, 1, 1)
4         _AmbientK ("Ambient", Range(0, 1)) = 0.1
5         _DiffuseK ("Diffuse", Range(0, 1)) = 0.7
6         _SpecularK ("Specular", Range(0, 1)) = 0.5
7         _Shininess ("Shininess", Range(1, 256)) = 32
8     }
9     SubShader {
10         Tags { "RenderPipeline"="UniversalPipeline" }
11         Pass {
12             Tags { "LightMode"="UniversalForward" }
13             HLSLPROGRAM
14             #pragma vertex vert
15             #pragma fragment frag
16             #include "Packages/com.unity.render-pipelines.universal
17                 /ShaderLibrary/Core.hlsl"
18             #include "Packages/com.unity.render-pipelines.universal
19                 /ShaderLibrary/Lighting.hlsl"
20
21             struct Attributes {
22                 float4 positionOS : POSITION;
23                 float3 normalOS : NORMAL;
24             };
25             struct Varyings {
26                 float4 positionCS : SV_POSITION;
27                 float3 normalWS : TEXCOORD0;
28                 float3 positionWS : TEXCOORD1;
29             };
30
31             float4 _Color;
32             float _AmbientK, _DiffuseK, _SpecularK, _Shininess;
33
34             Varyings vert(Attributes IN) {
35                 Varyings OUT;
36                 OUT.positionCS = TransformObjectToHClip(IN.positionOS.xyz);
37                 OUT.normalWS = TransformObjectToWorldNormal(IN.normalOS);
38                 OUT.positionWS = TransformObjectToWorld(IN.positionOS.xyz);
39                 return OUT;
40             }
41
42             float4 frag(Varyings IN) : SV_Target {
43                 // fragment shader diverso in base al modello
44             }
45             ENDDL
46         }
47     }
48 }
```

2.6.2 Modello di Phong

L'implementazione traduce direttamente l'equazione 2.6 in codice HLSL.

Listing 2.5: Fragment shader del modello di Phong.

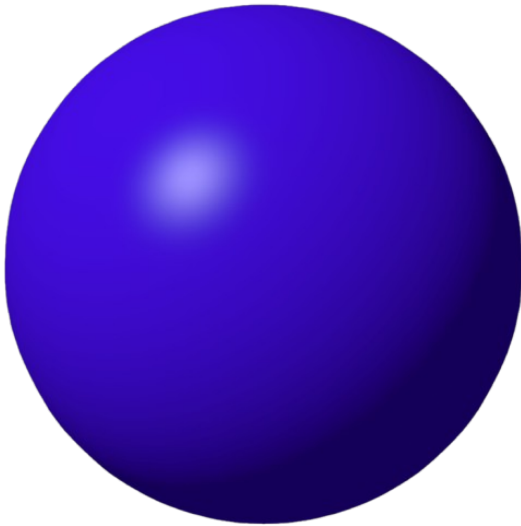
```
1 float4 frag(Varyings IN) : SV_Target {
2     Light light = GetMainLight(); // Sorgente luminosa
3     // Vettori
4     float3 N = normalize(IN.normalWS);
5     float3 L = normalize(light.direction);
6     float3 V = normalize(GetCameraPositionWS() - IN.positionWS);
7     float3 R = reflect(-L, N);
8     // Componenti
9     float3 ambient = _AmbientK * _Color.rgb;
10    float3 diffuse = _DiffuseK * max(dot(N, L), 0.0) * _Color.rgb;
11    float3 specular = _SpecularK * pow(max(dot(R, V), 0.0), _Shininess) * light.color;
12    // Risultato finale
13    return float4(ambient + diffuse + specular, 1.0);
14 }
```

2.6.3 Modello di Blinn-Phong

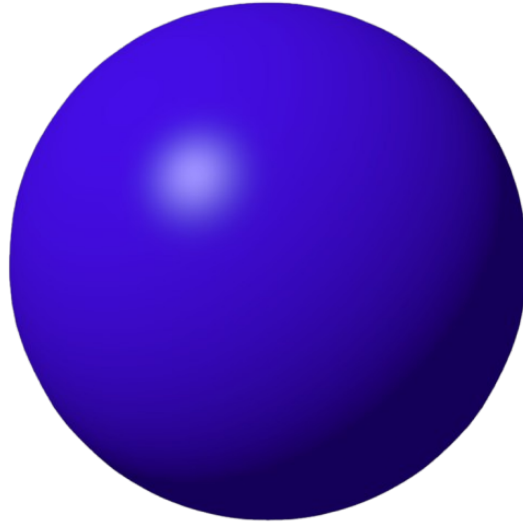
Rispetto al modello di Phong, l'unica differenza è il calcolo della componente speculare: il vettore di riflessione **R** viene sostituito dall'halfway vector **H**.

Listing 2.6: Fragment shader del modello di Blinn-Phong.

```
1 float4 frag(Varyings IN) : SV_Target {
2     Light light = GetMainLight(); // Sorgente luminosa
3     // Vettori
4     float3 N = normalize(IN.normalWS);
5     float3 L = normalize(light.direction);
6     float3 V = normalize(GetCameraPositionWS() - IN.positionWS);
7     float3 H = normalize(L + V);
8     // Componenti
9     float3 ambient = _AmbientK * _Color.rgb;
10    float3 diffuse = _DiffuseK * max(dot(N, L), 0.0) * _Color.rgb;
11    float3 specular = _SpecularK * pow(max(dot(N, H), 0.0), _Shininess * 4.0) *
        light.color;
12    // Risultato finale
13    return float4(ambient + diffuse + specular, 1.0);
14 }
```

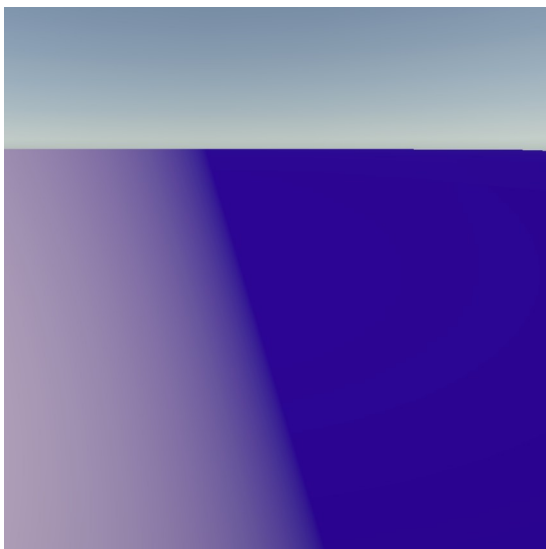


(a) Modello di Phong

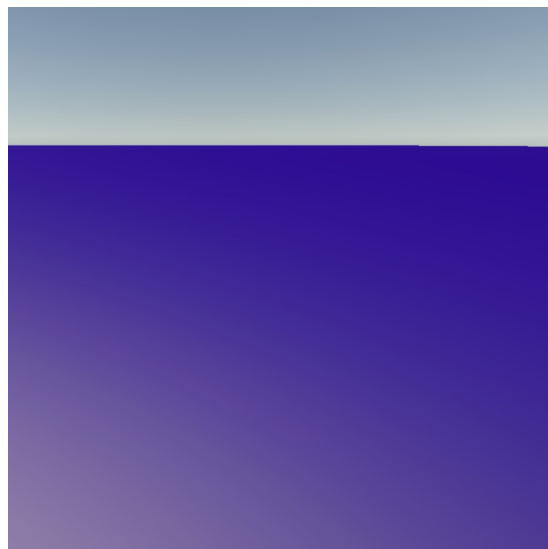


(b) Modello di Blinn-Phong

Figura 2.5: Confronto tra i modelli di Phong e Blinn-Phong con parametri identici.



(a) Modello di Phong



(b) Modello di Blinn-Phong

Figura 2.6: Riflesso speculare su un piano con luce radente e un valore di shininess basso. Nel modello di Phong il riflesso presenta un taglio netto, assente in Blinn-Phong.

Capitolo 3

Physically Based Rendering (PBR)

I modelli di Phong e Blinn-Phong, presentati nel capitolo precedente, sono modelli empirici: producono risultati visivamente plausibili, ma non si basano su principi fisici e i loro parametri vengono regolati manualmente, senza vincoli che ne garantiscano la correttezza. Il risultato è che, al variare delle condizioni di illuminazione, gli stessi materiali possono apparire convincenti in una scena e innaturali in un'altra.

Il *Physically Based Rendering* (PBR) nasce per superare queste limitazioni: anziché approssimare l'aspetto visivo con formule empiriche, si fonda su un modello fisico dell'interazione tra luce e materia basato sulla teoria delle microfacce. I materiali vengono descritti tramite grandezze fisicamente interpretabili (rugosità e metallicità) e viene rispettato il principio di conservazione dell'energia.

Questo capitolo presenta i principi fisici alla base del PBR, la BRDF di Cook-Torrance e i suoi termini costitutivi, il *workflow* metallic/roughness adottato nei motori moderni e un'introduzione all'*Image-Based Lighting*, concludendo con l'implementazione HLSL in Unity.

3.1 Principi fisici

Il PBR si fonda su alcuni principi fisici che ne vincolano il comportamento e lo distinguono dai modelli empirici.

3.1.1 Conservazione dell'energia

Il principio di conservazione dell'energia impone che la quantità totale di luce riflessa da una superficie non possa superare quella della luce incidente. Nei modelli classici questo vincolo non è garantito: i coefficienti k_a , k_d e k_s sono indipendenti tra loro e nulla impedisce che la loro combinazione produca una quantità di luce riflessa superiore a quella ricevuta. Nel PBR, invece, la luce incidente viene suddivisa in una componente riflessa e una che penetra nel materiale e viene diffusa. La formulazione del modello assicura che la somma di questi contributi non superi l'energia disponibile.

Di conseguenza, una superficie molto riflettente avrà necessariamente una componente diffusa debole, e viceversa. Questo produce un comportamento visivamente coerente: i materiali risultano convincenti sotto qualsiasi condizione di illuminazione, senza bisogno di regolazioni manuali.

3.1.2 Teoria delle microfacce

I modelli PBR descrivono una superficie non come un piano perfettamente liscio, ma come un insieme di piccole facce orientate in direzioni diverse, dette *microfacce*, ciascuna delle quali si comporta come uno specchio perfetto riflettendo la luce in un'unica direzione determinata dalla propria orientazione.

L'aspetto del materiale dipende dalla distribuzione degli orientamenti delle microfacce. Una superficie liscia presenta microfacce ben allineate: la luce viene riflessa quasi tutta nella stessa direzione, con un riflesso nitido e concentrato. Una superficie rugosa, invece, ha microfacce orientate caoticamente: la luce viene dispersa in molte direzioni, con un riflesso ampio e sfumato.

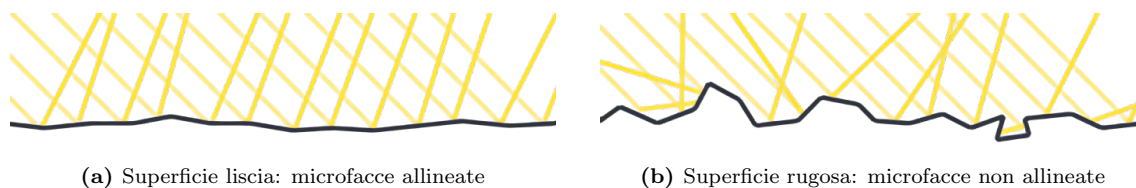


Figura 3.1: Rappresentazione delle microfacce. Immagini tratte da [dV].

Questa distribuzione viene descritta dal parametro *roughness* (rugosità), compreso tra 0 (superficie perfettamente liscia) e 1 (superficie completamente rugosa).

3.1.3 Riflessione e rifrazione

Quando la luce colpisce la superficie di un oggetto, una parte viene riflessa e una parte viene rifratta, penetrando nel materiale. La proporzione tra le due è descritta dalle *equazioni di Fresnel*, trattate nella Sezione 3.2.3.

Il comportamento della luce rifratta dipende dal tipo di materiale:

- Nei **dielettrici** (plastica, legno, ceramica) la luce rifratta viene diffusa internamente e riemerge in direzioni casuali, dando origine alla componente diffusa. Il riflesso speculare assume il colore della luce incidente, indipendentemente dal materiale.
- Nei **metalli** (oro, rame, alluminio) la luce rifratta viene assorbita e convertita in calore: la componente diffusa è quindi praticamente assente e tutta la risposta visiva è data dal riflesso speculare, il cui colore dipende sia dalle proprietà del metallo che dalla luce incidente.

Questa distinzione, assente nei modelli classici, viene modellata dal parametro *metallic*.

3.2 La BRDF di Cook-Torrance

Per descrivere come una superficie riflette la luce si utilizza una funzione chiamata *BRDF* (*Bidirectional Reflectance Distribution Function*). Essa definisce, data una

direzione di illuminazione $\mathbf{L}$ e una direzione di osservazione $\mathbf{V}$, la frazione di luce incidente che viene riflessa dalla superficie verso l'osservatore.

La BRDF non fornisce il colore finale di un punto: descrive solo il rapporto tra luce ricevuta e luce riflessa. Per ottenere l'illuminazione effettiva va moltiplicata per l'intensità della luce incidente L_i e per il coseno dell'angolo di incidenza $\mathbf{N} \cdot \mathbf{L}$, sommando il contributo di tutte le sorgenti presenti nella scena:

$$L_o = \sum_{\text{luci}} f_r \cdot L_i \cdot (\mathbf{N} \cdot \mathbf{L}) \quad (3.1)$$

dove L_o è l'intensità della luce riflessa verso l'osservatore, f_r è la BRDF, L_i è l'intensità della luce proveniente dalla direzione $\mathbf{L}$ e $\mathbf{N} \cdot \mathbf{L}$ è il coseno dell'angolo di incidenza, che riduce il contributo delle sorgenti che illuminano la superficie di taglio.

Questa è la forma semplificata della *rendering equation* per sorgenti luminose discrete^[Kaj86]. La componente ambientale non è inclusa: nel PBR viene gestita dall'Image-Based Lighting (Sezione 3.4) o approssimata con un termine costante.

Anche i modelli classici di Phong e Blinn-Phong possono essere considerati BRDF, sebbene empirici. Il PBR utilizza invece BRDF fisicamente accurate: quella più diffusa per il rendering in tempo reale è la BRDF di Cook-Torrance^[CT82], basata sulla teoria delle microfacce introdotta nella Sezione 3.1.2.

3.2.1 Struttura della BRDF

La BRDF di Cook-Torrance è composta da un termine diffuso e uno speculare:

$$f_r = f_{\text{diffuse}} + f_{\text{specular}} \quad (3.2)$$

La ripartizione dell'energia tra i due termini è garantita dal termine di Fresnel, come si vedrà nella Sezione 3.2.3.

Il termine diffuso f_{diffuse} è dato dal modello di Lambert (lo stesso utilizzato nei modelli classici) che descrive una superficie che riflette la luce uniformemente in tutte le direzioni:

$$f_{\text{diffuse}} = \frac{c}{\pi} \quad (3.3)$$

dove c è il colore del materiale (*albedo*). La divisione per π è un fattore di normalizzazione che garantisce la conservazione dell'energia. A differenza dei modelli classici, dove il coseno dell'angolo tra $\mathbf{N}$ e $\mathbf{L}$ viene incluso direttamente nella formula della componente diffusa, nel PBR non fa parte della BRDF ma viene introdotto separatamente nella rendering equation (3.1).

Il termine speculare f_{specular} costituisce il nucleo del modello ed è definito come:

$$f_{\text{specular}} = \frac{D \cdot F \cdot G}{4 (\mathbf{N} \cdot \mathbf{V}) (\mathbf{N} \cdot \mathbf{L})} \quad (3.4)$$

dove D , F e G modellano ciascuno un particolare fenomeno fisico:

- D (*normal distribution function*): descrive la distribuzione statistica degli orientamenti delle microfacce.
- F (*Fresnel*): descrive la proporzione di luce riflessa rispetto a quella rifratta in funzione dell'angolo di incidenza.
- G (*geometry function*): modella l'auto-ombreggiatura tra le microfacce.

Il denominatore $4 (\mathbf{N} \cdot \mathbf{V}) (\mathbf{N} \cdot \mathbf{L})$ è un fattore di normalizzazione derivato dal modello a microfacce.

Le sottosezioni successive descrivono nel dettaglio ciascuno dei tre termini.

3.2.2 Termine D (normal distribution function)

Il termine D è una funzione che descrive la distribuzione statistica degli orientamenti delle microfacce. Data una direzione $\mathbf{H}$ (halfway vector), D descrive la con-

centrazione di microfacce orientate in quella direzione, ovvero quelle che riflettono effettivamente la luce verso l'osservatore.

L'aspetto della superficie dipende da come queste microfacce sono distribuite. Su una superficie liscia (rugosità bassa) sono quasi tutte allineate alla normale: poche microfacce contribuiscono al riflesso, che risulterà piccolo e intenso. Su una superficie rugosa (rugosità alta) gli orientamenti sono molto più variabili: tante microfacce contribuiscono al riflesso, distribuendolo su un'area ampia ma con minore intensità.

La funzione D utilizzata nel modello di Cook-Torrance è la Trowbridge-Reitz (GGX)^[WMLT07], definita come:

$$D(\mathbf{N}, \mathbf{H}, \alpha) = \frac{\alpha^2}{\pi ((\mathbf{N} \cdot \mathbf{H})^2 (\alpha^2 - 1) + 1)^2} \quad (3.5)$$

dove $\mathbf{N}$ è la normale alla superficie, $\mathbf{H}$ è l'halfway vector (tra la direzione della sorgente luminosa e quella dell'osservatore) e α è la *roughness* del materiale. In molte implementazioni, tra cui quella adottata in questo progetto, si utilizza $\alpha = roughness^2$, in quanto produce una variazione percettivamente più lineare della rugosità al variare del parametro.

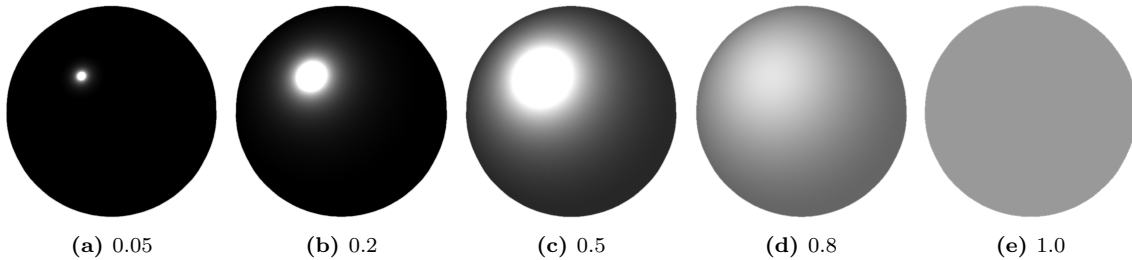


Figura 3.2: Visualizzazione del termine D al variare della rugosità. All'aumentare di α la distribuzione delle microfacce diventa più dispersa, producendo un riflesso più ampio e meno intenso.

3.2.3 Termine F (Fresnel)

Il termine F descrive l'effetto Fresnel: la proporzione di luce riflessa da una superficie, detta riflettanza, varia in funzione dell'angolo tra la direzione di osservazione $\mathbf{V}$ e la normale $\mathbf{N}$. In particolare, la riflettanza aumenta quanto più la superficie viene

osservata di taglio. Questo fenomeno è facilmente osservabile in natura: guardando uno specchio d'acqua dall'alto è spesso possibile vedere il fondo, mentre osservandolo in lontananza prevale il riflesso del paesaggio circostante.



Figura 3.3: Effetto Fresnel su un lago: in primo piano l'acqua appare trasparente, mentre in lontananza riflette il paesaggio.

Le equazioni di Fresnel originali sono complesse e poco pratiche per il calcolo in tempo reale. Nel modello di Cook-Torrance si utilizza l'approssimazione proposta da Schlick^[Sch94]:

$$F(\mathbf{H}, \mathbf{V}, F_0) = F_0 + (1 - F_0)(1 - (\mathbf{H} \cdot \mathbf{V}))^5 \quad (3.6)$$

dove $\mathbf{H}$ è l'halfway vector, $\mathbf{V}$ è la direzione dell'osservatore e F_0 è la *riflettanza base* del materiale, ovvero la frazione di luce riflessa quando l'osservatore guarda la superficie perpendicolarmente. Il valore di F_0 dipende dal tipo di materiale:

- per i dielettrici, F_0 è un valore basso e acromatico, tipicamente intorno a 0.04;

- per i metalli, F_0 è un valore alto e colorato, coincidente con il colore del materiale. È questo che produce il riflesso colorato dei metalli.

Materiale	F_0	Colore
Acqua	(0.02, 0.02, 0.02)	
Vetro	(0.03, 0.03, 0.03)	
Plastica	(0.05, 0.05, 0.05)	
Diamante	(0.17, 0.17, 0.17)	
Ferro	(0.56, 0.57, 0.58)	
Rame	(0.95, 0.64, 0.54)	
Oro	(1.00, 0.71, 0.29)	
Alluminio	(0.91, 0.92, 0.92)	
Argento	(0.95, 0.93, 0.88)	

Tabella 3.1: Valori di F_0 per alcuni materiali comuni. I dielettrici hanno valori bassi e acromatici, i metalli valori alti e colorati.

Il parametro *metallic* (con valori compresi tra 0 e 1) controlla l'interpolazione di F_0 tra il valore tipico dei dielettrici (0.04) e il colore dell'albedo del materiale, usato come F_0 per i metalli:

$$F_0 = \text{lerp}(0.04, \text{albedo}, \text{metallic}) \quad (3.7)$$

Il termine di Fresnel definisce anche la ripartizione dell'energia tra le componenti diffusa e speculare. La frazione di luce destinata alla componente diffusa è data da:

$$k_d = (1 - F)(1 - \text{metallic}) \quad (3.8)$$

Il fattore $(1 - F)$ garantisce la conservazione dell'energia: la porzione di luce non riflessa specularmente viene assegnata alla componente diffusa. Il termine $(1 - \text{metallic})$ annulla invece la componente diffusa nei materiali metallici, i quali riflettono la luce quasi esclusivamente in modo speculare.

Il termine diffuso della BRDF (equazione 3.2) diventa quindi:

$$f_{\text{diffuse}} = k_d \cdot \frac{c}{\pi} \quad (3.9)$$

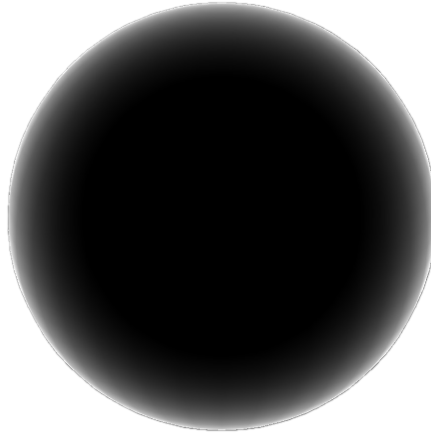


Figura 3.4: Visualizzazione del termine F su una sfera: la riflettanza aumenta alle zone periferiche, dove l'angolo tra $\mathbf{V}$ e $\mathbf{N}$ è maggiore.

3.2.4 Termine G (geometry function)

Il termine G modella il fenomeno di *auto-ombreggiatura* delle microfacce: su una superficie rugosa, alcune microfacce possono oscurare la luce in arrivo su quelle vicine (*shadowing*), mentre altre possono bloccare la luce riflessa prima che raggiunga l'osservatore (*masking*). Entrambi gli effetti riducono la quantità di luce effettivamente visibile e risultano tanto più rilevanti quanto più la superficie è rugosa.

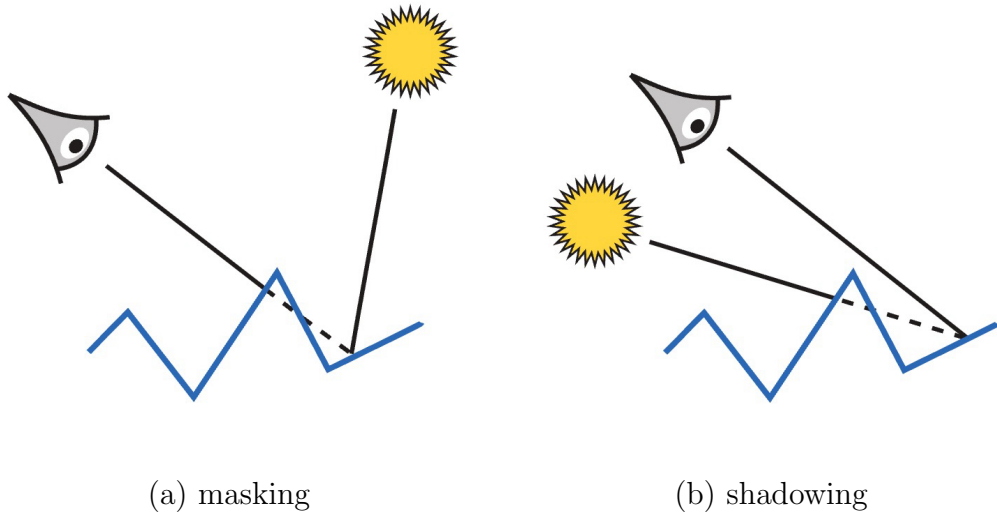


Figura 3.5: Nel masking (a) la microfaccia, pur orientata correttamente, non è visibile all'osservatore perché coperta da una microfaccia adiacente; nello shadowing (b) la luce non raggiunge la microfaccia perché bloccata da una vicina. Immagini tratte da ^[PJH23].

La funzione G combina i contributi di masking e shadowing tramite il *metodo di Smith*, che li valuta separatamente nelle direzioni dell'osservatore e della sorgente luminosa:

$$G(\mathbf{N}, \mathbf{V}, \mathbf{L}, k) = G_1(\mathbf{N}, \mathbf{V}, k) \cdot G_1(\mathbf{N}, \mathbf{L}, k) \quad (3.10)$$

dove $G_1(\mathbf{N}, \mathbf{V}, k)$ modella il masking, ossia la frazione di microfacce visibili all'osservatore, mentre $G_1(\mathbf{N}, \mathbf{L}, k)$ modella lo shadowing, cioè la frazione di microfacce raggiunte dalla luce. Il parametro k è una rimappatura della rugosità α , definita sotto.

Il prodotto dei due contributi risulta diverso da zero solo quando una microfaccia è contemporaneamente visibile all'osservatore e illuminata dalla sorgente luminosa.

Per ciascuno dei due fattori si utilizza l'approssimazione di Schlick-GGX:

$$G_1(\mathbf{N}, \mathbf{X}, k) = \frac{\mathbf{N} \cdot \mathbf{X}}{(\mathbf{N} \cdot \mathbf{X})(1 - k) + k} \quad (3.11)$$

dove $\mathbf{X}$ è un vettore generico sostituito rispettivamente con $\mathbf{V}$ e $\mathbf{L}$ per ottenere i due fattori della formula di Smith. Il parametro k è una rimappatura della rugosità α che dipende dal contesto: nel caso di luce diretta si utilizza:

$$k = \frac{(\alpha + 1)^2}{8} \quad (3.12)$$

L'effetto visivo di G è quello di attenuare le zone illuminate ad angoli radenti, in modo più marcato all'aumentare della rugosità.

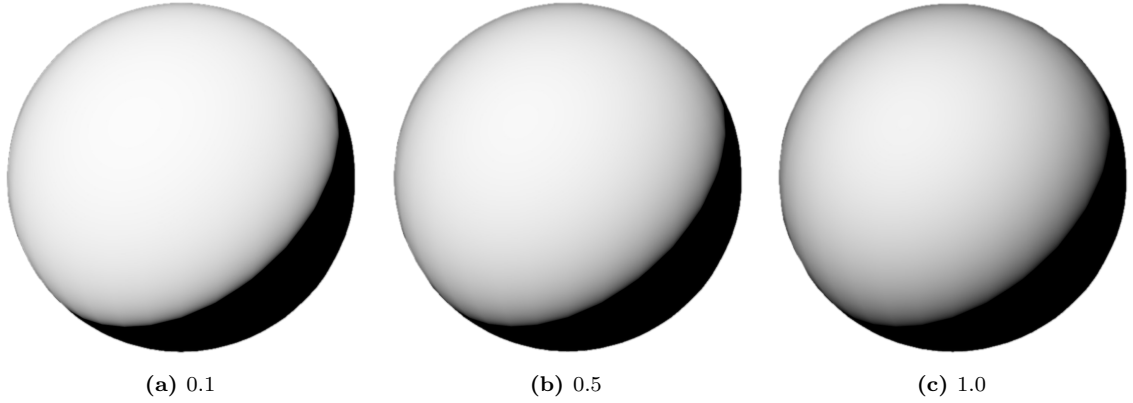


Figura 3.6: Visualizzazione del termine G al variare della rugosità. All'aumentare della rugosità, l'auto-ombreggiatura delle microfacce attenua maggiormente le zone illuminate a basso angolo di incidenza.

3.3 Metallic/Roughness workflow

I modelli PBR sono descritti in termini di grandezze fisiche: riflettanza base F_0 , rugosità delle microfacce, distinzione tra metalli e dielettrici. Per rendere questi parametri utilizzabili in pratica, i motori di rendering moderni adottano il *metallic/roughness workflow*, che riduce la configurazione di un materiale a pochi parametri intuitivi.

3.3.1 Parametri di un materiale

Un materiale PBR nel metallic/roughness workflow è descritto da tre parametri principali:

- **Albedo**: il colore base del materiale. Per i dielettrici rappresenta il colore della componente diffusa; per i metalli, dove la componente diffusa è assente, viene utilizzato come valore di F_0 , determinando il colore del riflesso speculare.
- **Metallic**: un valore compreso tra 0 e 1 che indica se il materiale è un dielettrico (0) o un metallo (1), controllando il valore di F_0 e la presenza della componente diffusa. Per un dielettrico si assume $F_0 = 0.04$ e la componente diffusa è presente; per un metallo F_0 coincide con l'albedo e la componente diffusa è assente. Questa distinzione viene realizzata tramite un'interpolazione:

$$F_0 = \text{lerp}(0.04, \text{albedo}, \text{metallic}) \quad (3.13)$$

- **Roughness**: la rugosità della superficie, compresa tra 0 (perfettamente liscia) e 1 (molto rugosa). Controlla il parametro α utilizzato nei termini D e G della BRDF.

Rispetto ai coefficienti k_a , k_d , k_s dei modelli classici, questi parametri hanno un significato fisico diretto e producono risultati coerenti senza necessità di regolazione manuale: la struttura del modello impedisce configurazioni che violino il principio di conservazione dell'energia.

3.3.2 Texture maps

Su un oggetto reale le caratteristiche del materiale variano da punto a punto (metalli arrugginiti, venature del legno, ...). Per rappresentare questa variazione, ogni parametro viene memorizzato in una texture che ne specifica il valore per ciascun pixel della superficie:

- **Albedo map**: definisce il colore base della superficie in ogni punto, indipendentemente dall'illuminazione.

- **Metallic map:** in scala di grigi, indica se un punto della superficie è dielettrico (valore 0) o metallico (valore 1). I valori intermedi sono rari e rappresentano superfici di transizione o materiali sporchi.
- **Roughness map:** in scala di grigi, controlla la rugosità della superficie: valori bassi producono riflessi nitidi e concentrati, valori alti riflessi ampi e diffusi.
- **Normal map:** simula dettagli geometrici senza aggiungere geometria reale, alterando la normale $\mathbf{N}$ punto per punto. Ogni pixel codifica quindi una direzione nello spazio, modificando il modo in cui la luce interagisce con la superficie e dando l'illusione di rilievi e cavità.

Le texture fungono quindi da semplice contenitore per i dati: il modello matematico sottostante rimane invariato, indipendentemente dal fatto che i parametri provengano da una texture o da valori costanti. Negli shader sviluppati in questa tesi vengono utilizzati valori costanti, per mantenere il codice semplice e concentrare l'attenzione sul modello di illuminazione.

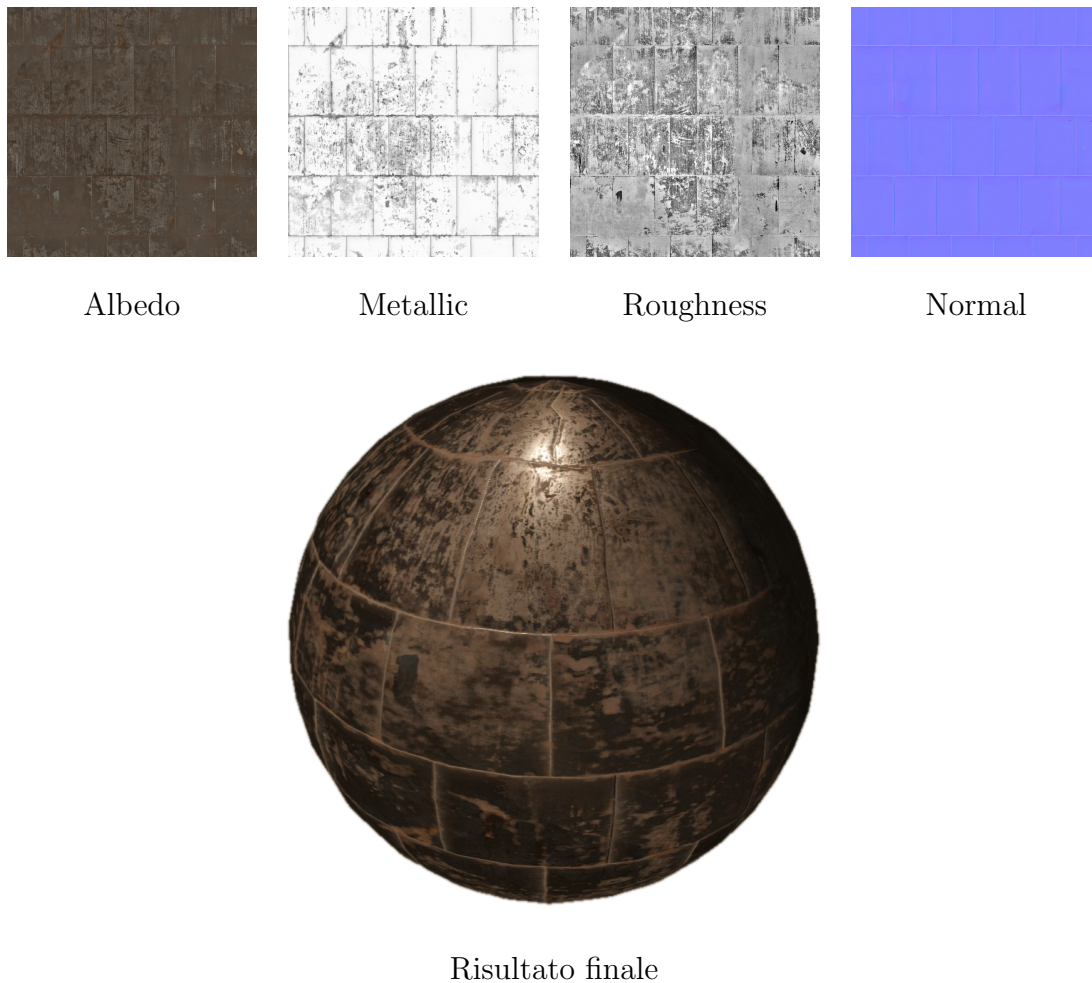


Figura 3.7: Le texture map e il risultato della loro combinazione nel modello di Cook-Torrance.

3.4 Image-Based Lighting

I modelli di illuminazione presentati finora considerano esclusivamente il contributo delle sorgenti luminose presenti nella scena, ovvero la luce diretta. Nella realtà, tuttavia, gli oggetti ricevono illuminazione anche dall'ambiente circostante: ogni superficie della scena contribuisce alla propagazione della luce. Questa componente, detta *luce indiretta*, è quella che nei modelli classici è approssimata mediante un termine ambientale costante.

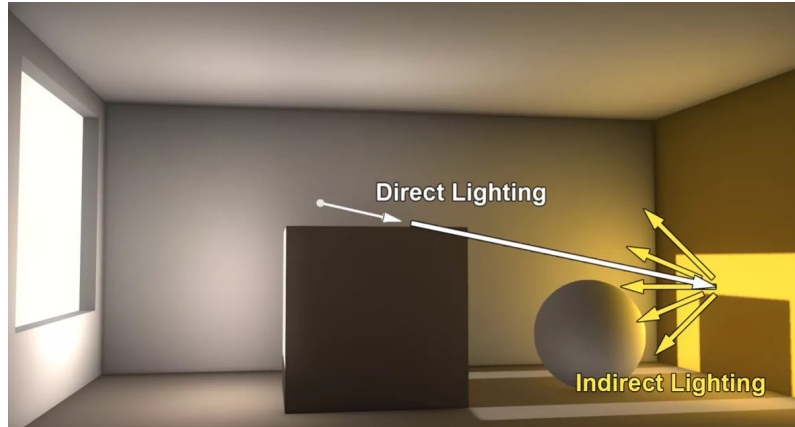


Figura 3.8: Confronto tra luce diretta e luce indiretta: la luce diretta raggiunge la superficie direttamente dalla sorgente, mentre la luce indiretta è il risultato dei rimbalzi successivi sulle superfici circostanti.

L'*Image-Based Lighting* (IBL) sostituisce questa approssimazione con un approccio più realistico, in cui l'ambiente circostante viene rappresentato tramite una *cubemap*, ossia una collezione di sei texture che descrivono la scena, utilizzata come sorgente luminosa per calcolare l'illuminazione indiretta su ogni punto della superficie.

L'IBL viene generalmente suddiviso in due contributi distinti:

- **Contributo diffuso:** rappresenta la luce ambientale proveniente da tutte le direzioni, ottenuta a partire dalla cubemap, detta *irradiance map*.
- **Contributo speculare:** rappresenta i riflessi dell'ambiente sulle superfici. L'aspetto del riflesso dipende dalla rugosità del materiale: superfici lisce producono riflessi nitidi, mentre superfici rugose producono riflessi più sfocati.

L'implementazione completa dell'IBL esula dagli obiettivi di questa tesi: negli shader sviluppati il contributo indiretto viene approssimato tramite un termine ambientale costante. Per un approfondimento sulla componente diffusa si rimanda a ^[dV14a], per quella speculare a ^[dV14b].



Figura 3.9: Una *cubemap* in formato a croce: le sei facce del cubo vengono "srotolate" in un'unica immagine e rappresentano l'ambiente circostante a 360°.

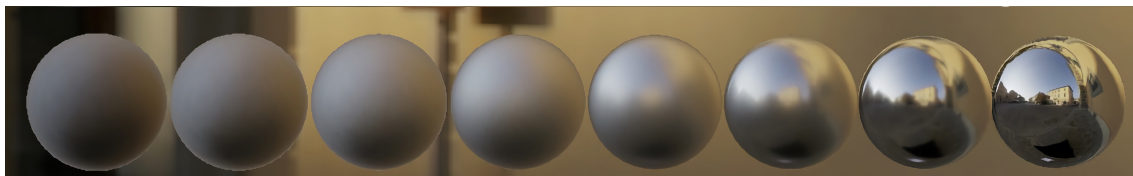


Figura 3.10: Sfere con rugosità crescente illuminate tramite IBL: al diminuire della rugosità i riflessi dell'ambiente diventano progressivamente più nitidi.

3.5 Implementazione HLSL del modello di Cook-Torrance

In questa sezione viene presentata l'implementazione HLSL del modello di Cook-Torrance, realizzata come shader custom per la Universal Render Pipeline di Unity.

3.5.1 Struttura dello shader

Lo shader espone all'inspector Unity i parametri del workflow metallic/roughness: albedo, metallic, roughness e un termine ambientale. La struttura del vertex shader

è identica a quella utilizzata per i modelli classici (Sezione 2.6.1).

Le tre funzioni D , F e G vengono implementate come funzioni HLSL separate, richiamate poi nel fragment shader per calcolare il colore finale.

3.5.2 Termine D

Implementazione della funzione 3.5:

Listing 3.1: Implementazione del termine D.

```
1 float D_GGX(float3 N, float3 H, float roughness) {  
2     float a = pow(roughness, 2.0);  
3     float NdotH = max(dot(N, H), 0.0);  
4     float denom = pow(NdotH, 2.0) * (pow(a, 2.0) - 1.0) + 1.0;  
5     return pow(a, 2.0) / (PI * pow(denom, 2.0));  
6 }
```

3.5.3 Termine F

Implementazione della funzione 3.6:

Listing 3.2: Implementazione del termine F.

```
1 float3 F_Schlick(float3 H, float3 V, float3 F0) {  
2     float HdotV = max(dot(H, V), 0.0);  
3     return F0 + (1.0 - F0) * pow(1.0 - HdotV, 5.0);  
4 }
```

3.5.4 Termine G

Implementazione delle funzioni 3.10 e 3.11:

Listing 3.3: Implementazione del termine G.

```
1 float G1_SchlickGGX(float3 N, float3 X, float roughness) {  
2     float NdotX = max(dot(N, X), 0.0);  
3     float k = pow(roughness + 1.0, 2.0) / 8.0;  
4     return NdotX / (NdotX * (1.0 - k) + k);  
5 }  
6  
7 float G_Smith(float3 N, float3 V, float3 L, float roughness) {  
8     return G1_SchlickGGX(N, V, roughness) * G1_SchlickGGX(N, L, roughness);  
9 }
```

3.5.5 Fragment shader

Il fragment shader assembla i tre termini nella BRDF di Cook-Torrance, combinando la componente speculare con quella diffusa di Lambert:

Listing 3.4: Fragment shader del modello Cook-Torrance.

```
1 float4 frag(Varyings IN) : SV_Target {
2     Light light = GetMainLight(); // Sorgente luminosa principale
3
4     // Vettori necessari
5     float3 N = normalize(IN.normalWS);
6     float3 L = normalize(light.direction);
7     float3 V = normalize(GetCameraPositionWS() - IN.positionWS);
8     float3 H = normalize(L + V);
9
10    float NdotL = max(dot(N, L), 0.0);
11    float NdotV = max(dot(N, V), 0.0);
12
13    // Calcolo F0
14    float3 F0 = lerp(0.04, _Albedo.rgb, _Metallic);
15
16    // Termini della BRDF
17    float D = D_GGX(N, H, _Roughness);
18    float3 F = F_Schlick(H, V, F0);
19    float G = G_Smith(N, V, L, _Roughness);
20
21    // Componente diffusa
22    float3 kD = (1.0 - F) * (1.0 - _Metallic);
23    float3 diffuse = kD * _Albedo.rgb / PI;
24
25    // Componente speculare
26    float3 specular = (D * F * G) / (4.0 * NdotV * NdotL + 0.0001);
27
28    // BRDF finale
29    float3 brdf = diffuse + specular;
30
31    // Risultato finale (rendering equation)
32    float3 color = brdf * light.color * NdotL;
33
34    // Approssimazione del termine ambientale
35    color += _AmbientK * _Albedo.rgb;
36
37    // Colore finale
38    return float4(color, 1.0);
39 }
```

3.5.6 Confronto tra Blinn-Phong e Cook-Torrance

Le immagini seguenti mettono a confronto il modello di Blinn-Phong e il modello di Cook-Torrance sullo stesso oggetto e nelle stesse condizioni di illuminazione.



(a) Blinn-Phong



(b) Cook-Torrance

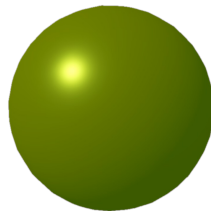
Figura 3.11: Confronto tra i modelli di Blinn-Phong e di Cook-Torrance nelle stesse condizioni di illuminazione.



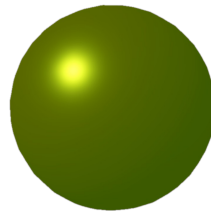
(a) $M=0$



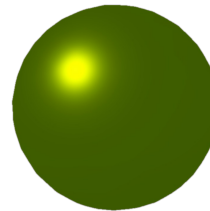
(b) $M=0.25$



(c) $M=0.5$



(d) $M=0.75$

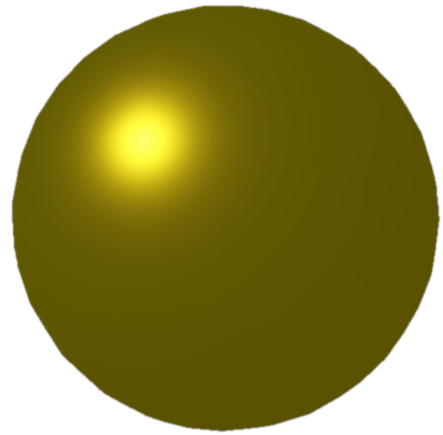


(e) $M=1$

Figura 3.12: Effetto del parametro metallic (roughness = 0.3). Al crescere di metallic la componente diffusa scompare e il riflesso speculare assume il colore dell'albedo.



(a) Blinn-Phong

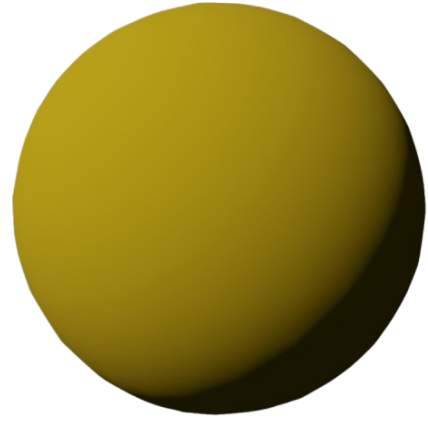


(b) Cook-Torrance

Figura 3.13: Confronto su un materiale metallico (oro). In Blinn-Phong il riflesso speculare è bianco, in Cook-Torrance assume il colore dell'albedo.



(a) Blinn-Phong



(b) Cook-Torrance

Figura 3.14: Confronto con rugosità elevata. In Cook-Torrance i bordi risultano più scuri per effetto del termine G (auto-ombreggiatura delle microfacce), assente in Blinn-Phong.

3.5.7 Supporto alle texture

L'implementazione presentata utilizza valori costanti per i parametri del materiale. Per supportare le texture è sufficiente sostituire le variabili uniformi con campionamenti delle texture corrispondenti nel fragment shader:

Listing 3.5: Lettura dei parametri da texture.

```
1 float3 albedo      = SAMPLE_TEXTURE2D(_AlbedoMap, sampler_AlbedoMap, IN.uv).rgb;
2 float metallic     = SAMPLE_TEXTURE2D(_MetallicMap, sampler_MetallicMap, IN.uv).r;
3 float roughness    = SAMPLE_TEXTURE2D(_RoughnessMap, sampler_RoughnessMap, IN.uv).r;
```

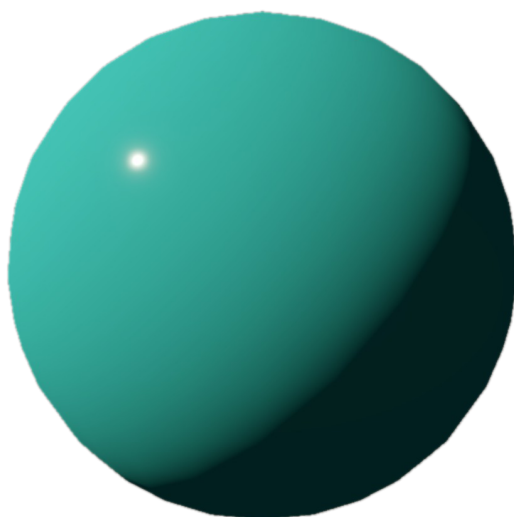
La normal map richiede un passaggio aggiuntivo. Le normali memorizzate nella texture sono espresse in un sistema di coordinate locale alla superficie (*tangent space*) e devono essere trasformate in world space per essere utilizzate nei calcoli di illuminazione.

Per farlo si costruisce la matrice **TBN** dai tre vettori che descrivono l'orientamento della superficie nel mondo: la tangente **T**, la bitangente **B** e la normale **N**, trasformati in world space dal vertex shader. La normale letta dalla texture viene moltiplicata per questa matrice, ottenendo una normale in world space che tiene conto dei dettagli simulati dalla normal map:

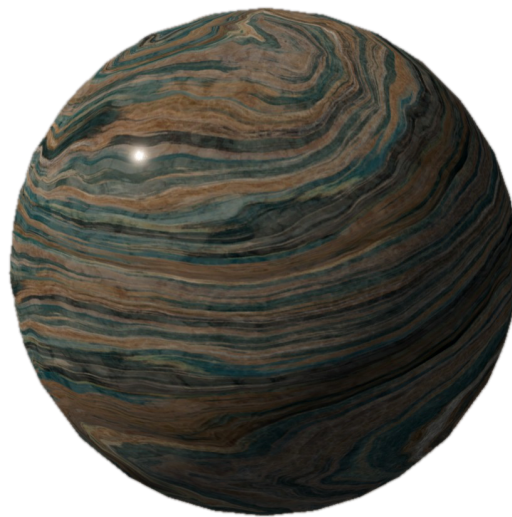
Listing 3.6: Applicazione della normal map (fragment shader).

```
1 // Lettura della normale dalla texture (in tangent space)
2 float3 normalTS =
3     UnpackNormal(SAMPLE_TEXTURE2D(_NormalMap, sampler_NormalMap, IN.uv));
4
5 // Matrice TBN
6 float3x3 TBN = float3x3(
7     normalize(IN.tangentWS),
8     normalize(IN.bitangentWS),
9     normalize(IN.normalWS)
10 );
11
12 // Normale modificata in world space
13 float3 N = normalize(mul(normalTS, TBN));
```

La normale **N** risultante sostituisce quella interpolata dal vertex shader, dando l'illusione di dettagli geometrici senza aggiungere geometria reale.



(a) Valori costanti



(b) Con texture maps

Figura 3.15: Il modello di Cook-Torrance con valori costanti e con texture maps. Le texture permettono di variare i parametri del materiale punto per punto.

Capitolo 4

Shader personalizzati per effetti non fotorealistici

Dopo aver analizzato modelli di illuminazione orientati alla resa realistica, questo capitolo mostra come la stessa pipeline programmabile possa essere utilizzata anche per ottenere effetti stilizzati e non fotorealistici. Gli shader presentati non hanno l'obiettivo di simulare fedelmente il comportamento fisico della luce, ma di evidenziare la flessibilità offerta da vertex e fragment shader nella definizione di modelli di resa personalizzati.

4.1 Toon shading

Il *toon shading* è una tecnica di rendering non fotorealistico che imita l'aspetto dei disegni animati e dei fumetti. Anziché produrre sfumature continue come i modelli classici, riduce l'illuminazione a poche fasce di colore con transizioni brusche tra zone illuminate e zone in ombra. Il modello di illuminazione alla base resta quello di Blinn-Phong, con la differenza che i valori calcolati vengono quantizzati in fasce discrete anziché applicati direttamente.

4.1.1 Componente diffusa quantizzata

Mentre nei modelli classici la componente diffusa varia in modo continuo, nel toon shading questo valore viene quantizzato, ovvero suddiviso in un numero discreto di fasce tramite la funzione *floor*:

$$I_{\text{toon}} = \frac{\lfloor (\mathbf{N} \cdot \mathbf{L}) \cdot s \rfloor}{s} \quad (4.1)$$

dove s è il numero di fasce.

4.1.2 Componente speculare con soglia

Anche la componente speculare viene stilizzata: anziché sfumare gradualmente con un esponente come nei modelli classici, viene ridotta a una scelta binaria basata su una soglia sul prodotto $\mathbf{N} \cdot \mathbf{H}$:

$$I_s = \begin{cases} k_s & \text{se } \mathbf{N} \cdot \mathbf{H} > t \\ 0 & \text{altrimenti} \end{cases} \quad (4.2)$$

dove t è la soglia e k_s l'intensità del riflesso. Il risultato è un riflesso dai bordi netti, tipico dello stile cartoon.

4.1.3 Linea di contorno

Per completare l'effetto fumetto, è possibile aggiungere un contorno nero attorno all'oggetto. La tecnica utilizzata si basa su un secondo pass di rendering: l'oggetto viene disegnato una prima volta leggermente ingrandito (estrudendo i vertici lungo le normali) e con le facce anteriori nascoste (`Cull Front`), mostrando solo le facce posteriori colorate di nero. Il pass principale disegna poi l'oggetto vero sopra, lasciando visibile il bordo nero come contorno.

4.1.4 Implementazione HLSL

L'implementazione è composta da due pass: il primo per la linea di contorno, il secondo per il toon shading.

Listing 4.1: Primo pass: estrude i vertici lungo la normale e disegna solo le facce posteriori.

```
1 Pass {
2     Cull Front // Nasconde le facce anteriori
3     HLSLPROGRAM
4
5     // ...
6
7     Varyings vert(Attributes IN) {
8         Varyings OUT;
9         float3 pos = IN.positionOS.xyz + IN.normalOS * _OutlineSize; // Estrusione
10        OUT.positionCS = TransformObjectToHClip(pos);
11        return OUT;
12    }
13
14    float4 frag(Varyings IN) : SV_Target {
15        return _OutlineColor; // Contorno di colore uniforme
16    }
17    ENDHLSL
18 }
```

Listing 4.2: Fragment shader del secondo pass: la componente diffusa viene quantizzata, mentre alla speculare viene applicata una soglia

```
1 float4 frag(Varyings IN) : SV_Target {
2     Light light = GetMainLight();
3
4     // Vettori
5     float3 N = normalize(IN.normalWS);
6     float3 L = normalize(light.direction);
7     float3 V = normalize(GetCameraPositionWS() - IN.positionWS);
8     float3 H = normalize(L + V);
9
10    // Componente diffusa quantizzata
11    float NdotL = max(dot(N, L), 0.0);
12    float toon = floor(NdotL * _Steps) / _Steps;
13
14    // Componente speculare con soglia
15    float NdotH = max(dot(N, H), 0.0);
16    float spec = NdotH > _SpecThresh ? _SpecularK : 0.0;
17
18    // Colore finale
19    float3 color = _Color.rgb * toon + spec * light.color;
20    return float4(color, 1.0);
21 }
```

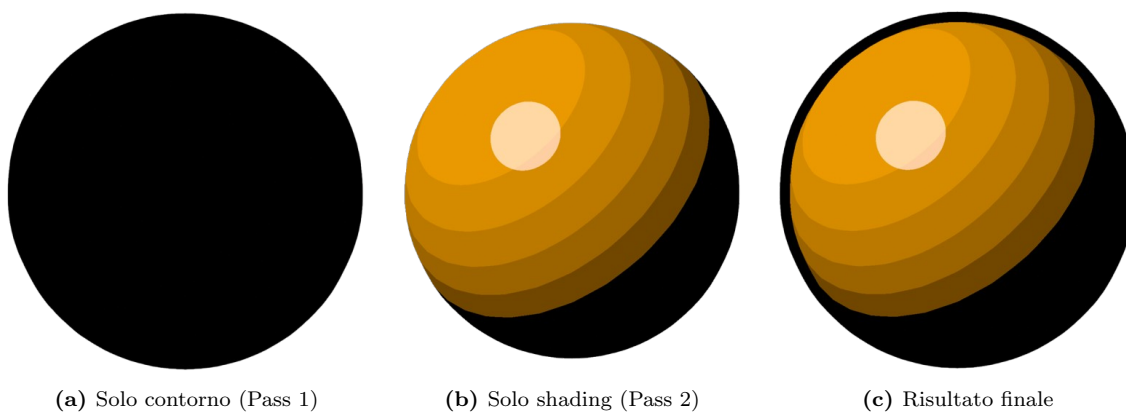


Figura 4.1: I due pass del toon shading e il risultato della loro combinazione.

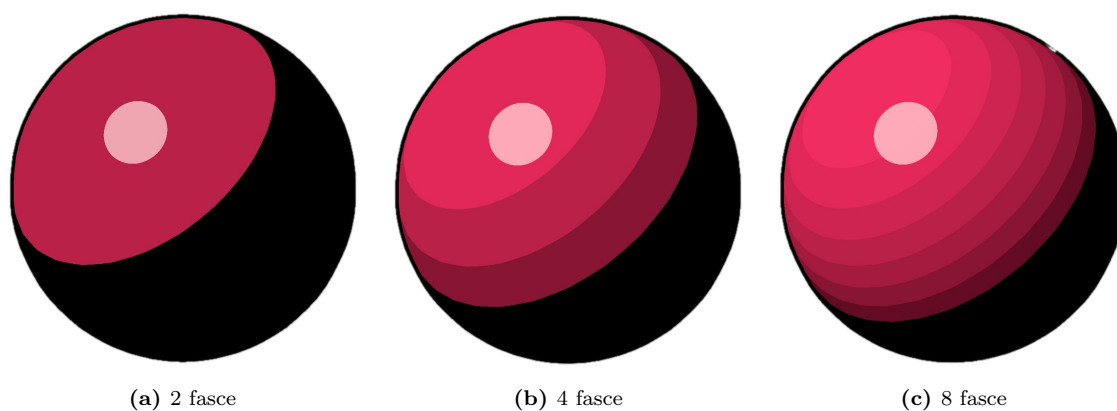


Figura 4.2: Effetto del numero di fasce nel toon shading. Con poche fasce l'aspetto è più stilizzato, con più fasce la transizione è più graduale.

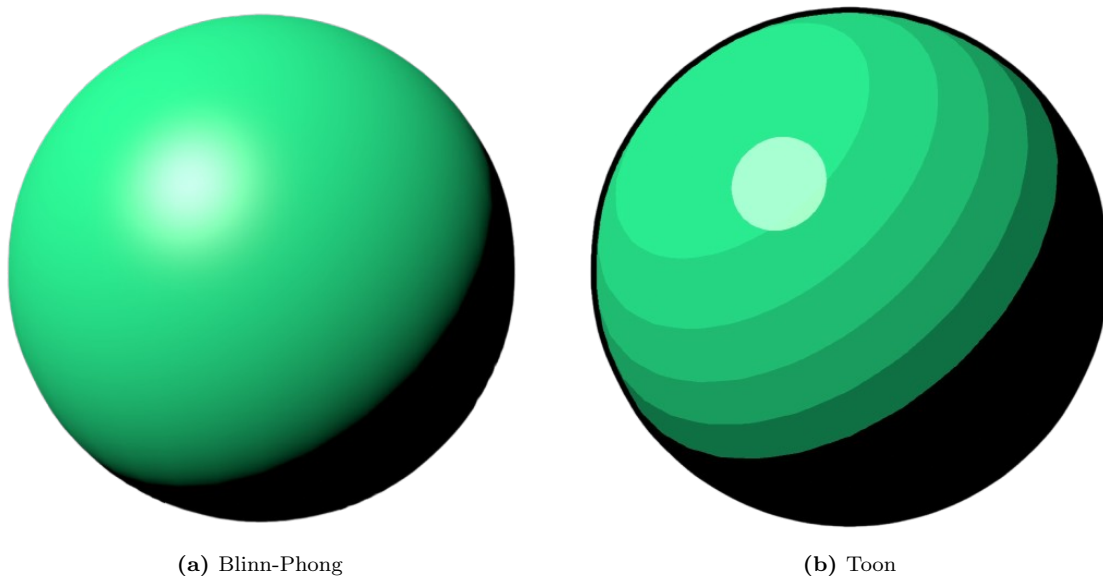


Figura 4.3: Confronto tra Blinn-Phong e toon shading sullo stesso oggetto.

4.2 Hologram shader

Questo shader non si basa su un modello di illuminazione fisico, ma combina tre effetti sovrapposti per simulare l'aspetto di un ologramma: *rim lighting*, pattern a strisce e *flicker*.

4.2.1 Rim lighting

Il *rim lighting* illumina i bordi dell'oggetto sfruttando lo stesso principio dell'effetto Fresnel: il prodotto $\mathbf{N} \cdot \mathbf{V}$ misura quanto la superficie è rivolta verso l'osservatore. Ai bordi, dove la superficie è quasi perpendicolare alla direzione di vista, questo valore tende a zero e il contributo del rim lighting al massimo:

$$I_{\text{rim}} = (1 - \mathbf{N} \cdot \mathbf{V})^p \quad (4.3)$$

dove p controlla la concentrazione dell'effetto: valori alti producono bordi più sottili e intensi.

4.2.2 Pattern a strisce

Questo effetto consiste in bande orizzontali che scorrono verso il basso lungo la superficie. Si ottiene applicando una funzione sinusoidale alla coordinata verticale y del frammento, animata nel tempo:

$$\text{scan} = \begin{cases} 1 & \text{se } \sin(y \cdot d + t \cdot v) > 0 \\ 0 & \text{altrimenti} \end{cases} \quad (4.4)$$

dove d è la densità delle strisce, v la velocità di scorrimento e t il tempo.

4.2.3 Flicker

Il *flicker* simula l'instabilità dell'ologramma modulando l'intensità luminosa con una senoide nel tempo:

$$I_{\text{flicker}} = 0.95 + 0.05 \cdot \sin(t \cdot f) \quad (4.5)$$

dove f è la frequenza del tremolio e t il tempo. Il valore oscilla tra 0.90 e 1.00.

4.2.4 Trasparenza

A differenza degli shader precedenti, questo richiede la trasparenza: l'oggetto deve apparire semi-trasparente. In Unity si ottiene abilitando l'*alpha blending*, che combina il colore di ogni frammento con quello già presente nel framebuffer in base al valore alpha, e impostando la coda di rendering a `Transparent`:

Listing 4.3: Configurazione per la trasparenza.

```
1 Tags {  
2     "RenderPipeline"="UniversalPipeline"  
3     "Queue"="Transparent "  
4     "RenderType"="Transparent "  
5 }  
6 Blend SrcAlpha OneMinusSrcAlpha  
7 ZWrite Off
```

ZWrite Off disabilita la scrittura nel depth buffer, necessaria per evitare che le parti trasparenti dell'oggetto nascondano la geometria retrostante.

4.2.5 Implementazione HLSL

Il fragment shader combina i tre effetti e calcola l'opacità finale:

Listing 4.4: Fragment shader dell'effetto ologramma.

```
1 float4 frag(Varyings IN) : SV_Target {
2     float3 N = normalize(IN.normalWS);
3     float3 V = normalize(GetCameraPositionWS() - IN.positionWS);
4
5     // Rim lighting
6     float rim = pow(1.0 - max(dot(N, V), 0.0), _RimPower);
7
8     // Pattern a strisce
9     float scan = sin(IN.positionWS.y * _ScanDensity + _Time.y * _ScanSpeed);
10    scan = step(0.0, scan);
11
12    // Flicker
13    float flicker = 0.95 + 0.05 * sin(_Time.y * _FlickerSpeed);
14
15    // Opacita'
16    float alpha = (rim + _ScanAlpha * scan) * flicker;
17    alpha = clamp(alpha, 0.1, 1.0);
18
19    // Colore finale
20    float3 color = _Color.rgb * (rim + 0.3) * flicker;
21
22    return float4(color, alpha);
23 }
```

L'opacità è data dalla somma del rim lighting e del pattern a strisce, modulata dal flicker. Il clamp garantisce un'opacità minima di 0.1, evitando che l'oggetto diventi completamente invisibile.

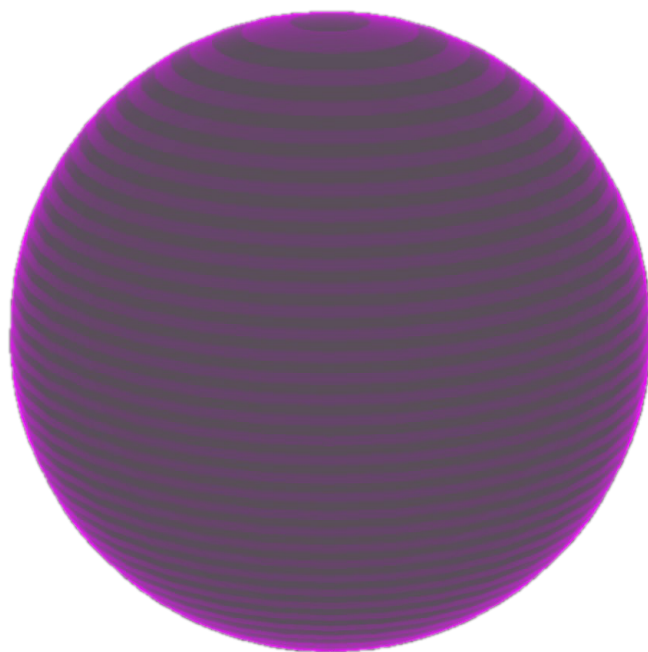


Figura 4.4: Effetto ologramma applicato a un oggetto. I bordi appaiono più luminosi e opachi, le bande orizzontali scorrono verticalmente e un leggero tremolio simula l'instabilità della proiezione.

Gli shader descritti in questo capitolo, insieme ai modelli di illuminazione classici e al modello PBR presentati nei capitoli precedenti, sono stati integrati nel progetto dimostrativo sviluppato in Unity. Il capitolo successivo descrive la struttura del progetto, le scelte implementative adottate e le modalità con cui i diversi shader possono essere confrontati all'interno della scena.

Capitolo 5

Progetto dimostrativo in Unity

Per verificare nella pratica le differenze tra i modelli di illuminazione analizzati e permettere un confronto diretto e interattivo, è stato realizzato un progetto dimostrativo in Unity. L'applicazione consente di esplorare una scena in prima persona, selezionare i singoli oggetti e modificarne in tempo reale lo shader e i relativi parametri, osservando l'effetto delle modifiche.

5.1 Struttura della scena

La scena contiene diversi oggetti importati in formato FBX, disposti in un ambiente esplorabile e configurati con materiali predefiniti, scelti per mostrare il comportamento degli shader su geometrie di diversa natura.

L'illuminazione è affidata a una singola luce direzionale, i cui parametri (intensità, direzione e colore) possono essere regolati a runtime. La scena è completata da uno skybox.



Figura 5.1: Panoramica della scena dimostrativa.

5.2 Navigazione in prima persona

La navigazione nella scena avviene in prima persona: il movimento è controllato dai tasti WASD e l'orientamento della visuale dal mouse. Il controllo è implementato nello script `FirstPersonCamera`, applicato alla camera. La rotazione verticale (*pitch*), limitata a $\pm 90^\circ$ per evitare ribaltamenti, viene applicata direttamente alla camera, mentre la rotazione orizzontale (*yaw*) viene applicata all'oggetto genitore. Il movimento utilizza un `CharacterController`, che gestisce automaticamente le collisioni con la geometria della scena.

Listing 5.1: Logica di movimento e rotazione della camera.

```
1 // Rotazione visuale con mouse
2 float mouseX = Input.GetAxis("Mouse X") * sensitivity;
3 float mouseY = Input.GetAxis("Mouse Y") * sensitivity;
4
5 rotX -= mouseY;
6 // Rotazione verticale limitata
7 rotX = Mathf.Clamp(rotX, -90f, 90f);
8 // Rotazione verticale applicata alla camera
9 transform.localRotation = Quaternion.Euler(rotX, 0f, 0f);
10 // Rotazione orizzontale applicata al player
11 transform.parent.Rotate(Vector3.up * mouseX);
12
13 // Movimento con WASD
14 float h = Input.GetAxis("Horizontal");
15 float v = Input.GetAxis("Vertical");
16 Vector3 move = transform.parent.right * h + transform.parent.forward * v;
17 // Movimento del player
18 controller.Move(move * moveSpeed * Time.deltaTime);
```

5.3 Selezione degli oggetti

Ogni oggetto modificabile è dotato dello script `SelectableObject`, che ne definisce il nome e l'insieme di materiali disponibili. All'avvio, lo script crea una copia di ciascun materiale, in modo che le modifiche apportate a runtime siano indipendenti per ogni oggetto e non alterino i materiali originali. Il primo materiale dell'elenco viene applicato come predefinito.

Poiché i modelli importati possono essere composti da più oggetti annidati, lo script raccoglie tutti i `Renderer` (il componente Unity responsabile del disegno di una mesh a schermo) presenti nell'oggetto e nei suoi figli applicando il materiale a ciascuno di essi.

Listing 5.2: Creazione delle istanze dei materiali e applicazione.

```
1 void Awake() {
2     // I Renderer dell'oggetto
3     renderers = GetComponentsInChildren<Renderer>();
4     // Copia dei materiali
5     instances = new Material[materials.Length];
6     for (int i = 0; i < materials.Length; i++)
7         instances[i] = new Material(materials[i]);
8     // Applica il primo materiale della lista
9     if (instances.Length > 0)
10         ApplyMaterial(instances[0]);
11 }
12
13 public void ApplyMaterial(Material mat) {
14     // Applica un materiale all'oggetto
15     foreach (var r in renderers)
16         r.material = mat;
17 }
```

La selezione di un oggetto avviene tramite *raycasting*: al click del mouse viene lanciato un raggio dal centro dello schermo, e il primo oggetto colpito dotato di `SelectableObject` viene selezionato. Un mirino al centro dello schermo indica il punto di mira.

Listing 5.3: Selezione di un oggetto tramite raycast.

```
1 // Raggio dal centro dello schermo
2 Ray ray = Camera.main.ScreenPointToRay(new Vector3(Screen.width/2, Screen.height/2));
3 RaycastHit hit;
4 if (Physics.Raycast(ray, out hit, selectDistance)) {
5     var obj = hit.collider.GetComponentInParent<SelectableObject>();
6     if (obj != null) {
7         selected = obj;
8         // apertura del pannello
9     }
10 }
```

5.4 Pannello selezione dello shader

Una volta selezionato un oggetto, si apre un pannello che permette di modificare shader e parametri dei materiali. Il pannello è realizzato con il sistema *IMGUI* di Unity, scelto per la sua semplicità: non richiede la costruzione di un'interfac-

cia nell'editor e permette di generare dinamicamente i controlli in base allo shader selezionato.

Il pannello mostra:

- il nome dell'oggetto selezionato;
- l'elenco degli shader disponibili per quell'oggetto, selezionabili;
- i parametri del materiale corrente, generati dinamicamente: per ogni proprietà dello shader (roughness, metallic, shininess, colore, ...) viene mostrato il controllo corrispondente solo se presente;
- i parametri della luce direzionale (intensità, rotazione, colore).

La generazione dinamica dei controlli si basa sul metodo `HasProperty`, che verifica la presenza di una proprietà nello shader prima di mostrare il relativo slider:

Listing 5.4: Esempio di generazione di un controllo del pannello.

```
1 if (mat.HasProperty("_Metallic"))  
2     mat.SetFloat(  
3         "_Metallic",  
4         Slider("Metallic", mat.GetFloat("_Metallic"), 0, 1)  
5     );
```

In questo modo lo stesso pannello si adatta automaticamente a qualsiasi shader: un materiale Blinn-Phong mostrerà i controlli per ambient, diffuse, specular e shininess, mentre un materiale PBR mostrerà metallic e roughness.

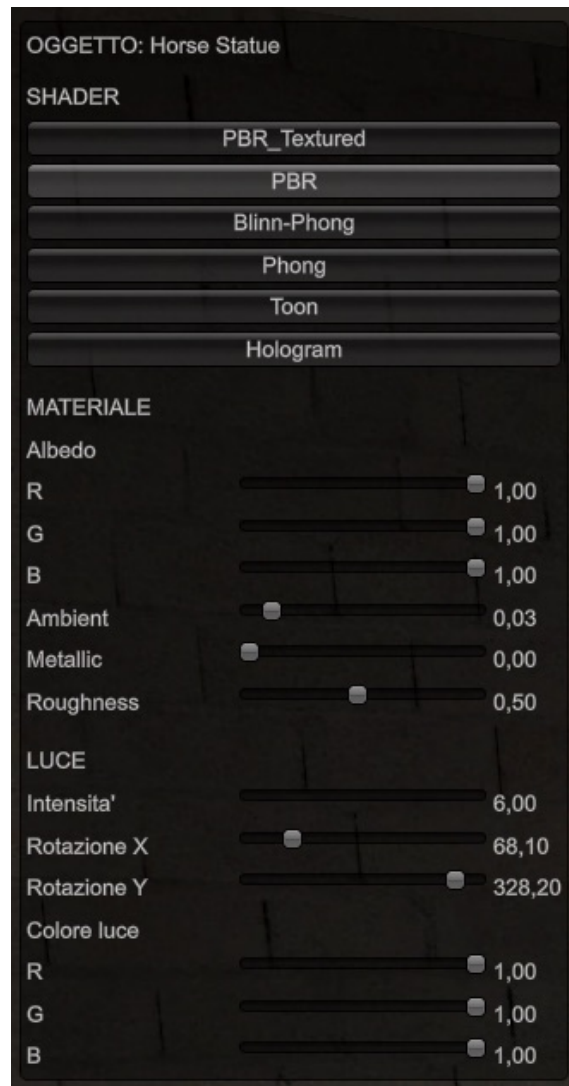


Figura 5.2: Il pannello di selezione dello shader aperto su un oggetto selezionato.

5.5 Confronto visivo tra i modelli di illuminazione

Il progetto permette di applicare a ciascun oggetto tutti i modelli implementati nei capitoli precedenti, confrontandoli direttamente nella stessa scena e sotto la stessa illuminazione. Le immagini seguenti mostrano lo stesso oggetto con i diversi shader.



(a) Phong



(b) Blinn-Phong



(c) PBR



(d) PBR con texture



(e) Toon



(f) Hologram

Figura 5.3: Lo stesso oggetto con i sei shader implementati, nella stessa scena e con la stessa illuminazione.

5.6 Build dell'applicazione

Il progetto è stato infine esportato come applicazione eseguibile tramite il sistema di build di Unity, generando una versione indipendente dall'editor. In questo modo la demo può essere eseguita direttamente, senza necessità di aprire il progetto in Unity, ed è utilizzabile per un confronto interattivo dei modelli implementati.

Capitolo 6

Conclusioni

Questo elaborato ha presentato i principali modelli di illuminazione utilizzati nella grafica in tempo reale, dai modelli empirici classici fino al Physically Based Rendering, accompagnando la trattazione teorica con implementazioni concrete in HLSL all'interno del motore Unity.

6.1 Riepilogo dei risultati

Sono stati implementati e confrontati diversi modelli di illuminazione — Phong, Blinn-Phong e Cook-Torrance — ciascuno realizzato come shader personalizzato per la Universal Render Pipeline di Unity. Il confronto ha evidenziato come il passaggio dai modelli classici al PBR rappresenti un cambio di approccio: i modelli classici approssimano l'aspetto della luce con formule empiriche i cui parametri vengono regolati manualmente, mentre il PBR si fonda su principi fisici che garantiscono per costruzione la coerenza visiva dei materiali.

Le differenze più significative emerse dal confronto sono l'effetto Fresnel, che modifica la riflettanza in base all'angolo di osservazione, la conservazione dell'energia, che impedisce configurazioni fisicamente impossibili, e la distinzione tra metalli e dielettrici, assente nei modelli classici.

Sono stati inoltre realizzati due shader personalizzati — effetto toon e ologramma — che dimostrano come la pipeline programmabile non sia limitata al fotorealismo, ma permetta di implementare qualsiasi modello di illuminazione o effetto visivo.

Il progetto in Unity ha permesso di confrontare tutti i modelli in tempo reale sulla stessa scena, verificando nella pratica le differenze teoriche analizzate nei capitoli precedenti.

6.2 Limiti del lavoro

L'implementazione presentata si concentra sulla luce diretta proveniente da una singola sorgente direzionale. Il contributo della luce indiretta, normalmente gestito tramite Image-Based Lighting, è stato approssimato con un termine ambientale costante. Questa semplificazione, pur sufficiente a dimostrare il funzionamento del modello, limita la resa visiva dei materiali, in particolare dei metalli, che senza riflessi ambientali appaiono più scuri di quanto sarebbero in un ambiente reale. Gli shader non gestiscono inoltre ombre proiettate o luci multiple.

6.3 Possibili sviluppi futuri

Il lavoro presentato potrebbe essere esteso in diverse direzioni:

- **Luci multiple e ombre:** il supporto a più sorgenti luminose e alle ombre proiettate renderebbe gli shader utilizzabili in scene più complesse e realistiche.
- **Image-Based Lighting:** un'implementazione completa dell'IBL migliorerebbe significativamente la resa dei materiali, in particolare dei metalli.
- **Modelli avanzati:** BRDF più sofisticate permetterebbero di rappresentare una gamma più ampia di materiali.
- **Ray tracing:** le GPU moderne supportano il ray tracing in tempo reale, che permette di calcolare riflessioni, rifrazioni e illuminazione globale in modo fisicamente accurato, superando le approssimazioni del rendering rasterizzato.

Riferimenti bibliografici

- [Bli77] James F. Blinn. Models of light reflection for computer synthesized pictures. *ACM SIGGRAPH Computer Graphics*, 11(2):192–198, 1977.
- [CT82] Robert L. Cook and Kenneth E. Torrance. A reflectance model for computer graphics. *ACM Transactions on Graphics*, 1(1):7–24, 1982.
- [dV] Joey de Vries. Pbr — theory. <https://learnopengl.com/PBR/Theory>.
- [dV14a] Joey de Vries. Diffuse irradiance. <https://learnopengl.com/PBR/IBL/Diffuse-irradiance>, 2014.
- [dV14b] Joey de Vries. Specular ibl. <https://learnopengl.com/PBR/IBL/Specular-IBL>, 2014.
- [Gou71] Henri Gouraud. Continuous shading of curved surfaces. *IEEE Transactions on Computers*, C-20(6):623–629, 1971.
- [Kaj86] James T. Kajiya. The rendering equation. In *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques*, pages 143–150, 1986.
- [Pho75] Bui Tuong Phong. Illumination for computer generated pictures. *Communications of the ACM*, 18(6):311–317, 1975.
- [PJH23] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation*. MIT Press, 4 edition, 2023.

- [Sch94] Christophe Schlick. An inexpensive brdf model for physically-based rendering. *Computer Graphics Forum*, 13(3):233–246, 1994.
- [Wik] Wikipedia. Blinn–phong reflection model. https://en.wikipedia.org/wiki/Phong_reflection_model.
- [WMLT07] Bruce Walter, Stephen R. Marschner, Hongsong Li, and Kenneth E. Torrance. Microfacet models for refraction through rough surfaces. In *Proceedings of the 18th Eurographics Conference on Rendering Techniques*, pages 195–206, 2007.

Ringraziamenti

ringraziamenti